Chapter 7: Modeling the Event

Powerful research questions can be unpacked as soon as we have the tools for counting mentions of words over time. Previously, we've been unpacking research questions that stem from the tools we have – like the ability to follow particular speakers or words through time. But another approach, which leverages the power of historical analysis, is to start with the event and ask: what can computational approaches teach us about history that we previously did not know?

In the history of parliament, one of the most important events is the Reform Act of 1832, which granted the middle class the right to vote, and the Reform Act of 1867, which granted the right to vote to most working-class men living in cities.

To analyze how the reform acts changed parliament, we can compare the speakers and speeches a decade before and a decade after each event. Such an approach can be accomplished with the tools that have already been taught in previous chapters. However, it requires us to load three decades of data at a time.

0.0.1 Working With Big Data: Cautions Before Proceeding

In the previous chapters, we worked with intentionally small, subsetted datasets so the examples would run on a laptop. In this chapter, we take the opposite approach: we join multiple decades and create a much larger dataset first. We then filter it as needed. Doing so, we demonstrate another workflow for working with large-scale historical data on a computer with limited resources. This time, the analyst begin with the full corpus and progressively narrow the scope through filtering and subsetting.

Previously, we demonstrated techniques were we worked with chunks of the data at a time to save computer memory. We chunked the data so the code will be able to process it on a laptop.[1]

```
# load libraries for cleaning, analyzing and visualizing data
library(tidyverse)
library(tidytext)
library(lubridate)

# load libraries for measurement and the hansard corpus
library(dhmeasures)
library(hansardr)

# load libraries for formatting and styling plots
library(scales)
library(ggrepel)
library(kableExtra)

# load the hansard debate text
```

```r
data("hansard_1820")
data("hansard_1830")
data("hansard_1840")

# load the hansard debate metadata
data("debate_metadata_1820")
data("debate_metadata_1830")
data("debate_metadata_1840")

# load the hansard speaker metadata
data("speaker_metadata_1820")
data("speaker_metadata_1830")
data("speaker_metadata_1840")
```

```r
# Add metadata to the 1820s Hansard data
# - left_join() adds debate and speaker metadata
# - mutate() extracts the year from the speech date
# - select() keeps only the relevant columns
annotated_1820 <- hansard_1820 %>%
  left_join(debate_metadata_1820) %>% # Add debate metadata
  mutate(year = year(speechdate)) %>% # Extract the year
  left_join(speaker_metadata_1820) %>% # Add speaker metadata
  select(text, year, speechdate, suggested_speaker) # Keep the relevant columns

# Do the same for the 1830s Hansard data
annotated_1830 <- hansard_1830 %>%
  left_join(debate_metadata_1830) %>%
  mutate(year = year(speechdate)) %>%
  left_join(speaker_metadata_1830) %>%
  select(text, year, speechdate, suggested_speaker)

# Do the same for the 1840s Hansard data
annotated_1840 <- hansard_1840 %>%
  left_join(debate_metadata_1840) %>%
  mutate(year = year(speechdate)) %>%
  left_join(speaker_metadata_1840) %>%
  select(text, year, speechdate, suggested_speaker)
```

```r
# Create the prereform dataset
# - bind_rows() combines the 1820s and 1830s data
# - filter() keeps speeches between 1822 and the Second Reform Act
# - mutate() labels the time period
prereform <- bind_rows(annotated_1820, annotated_1830) %>%
  filter(speechdate >= as.Date("1822-06-04"), # Keep speeches on or after June 4, 1822
         speechdate <= as.Date("1832-06-04")) %>% # Keep speeches on or before June 4, 1832
  mutate(period = "prereform") # Label the period
```

```
# Create the postreform dataset
# - bind_rows() combines the 1830s and 1840s data
# - filter() keeps speeches between the Second Reform Act and 1842
# - mutate() labels the time period
postreform <- bind_rows(annotated_1830, annotated_1840) %>%
  filter(speechdate >= as.Date("1832-06-04"), # Keep speeches on or after June 4, 1832
         speechdate <= as.Date("1842-06-04")) %>% # Keep speeches on or before June 4, 1842
  mutate(period = "postreform") # Label the period
```

`hansard` is a large dataset that includes three decades of parliamentary debates along with multiple layers of metadata that make the text analyzable. The debate metadata (which includes information such as the date of the speech and the title of the debate) and the speaker metadata (which includes the original speaker name and the disambiguated canonical name) are linked to each sentence through a shared `sentence_id`.

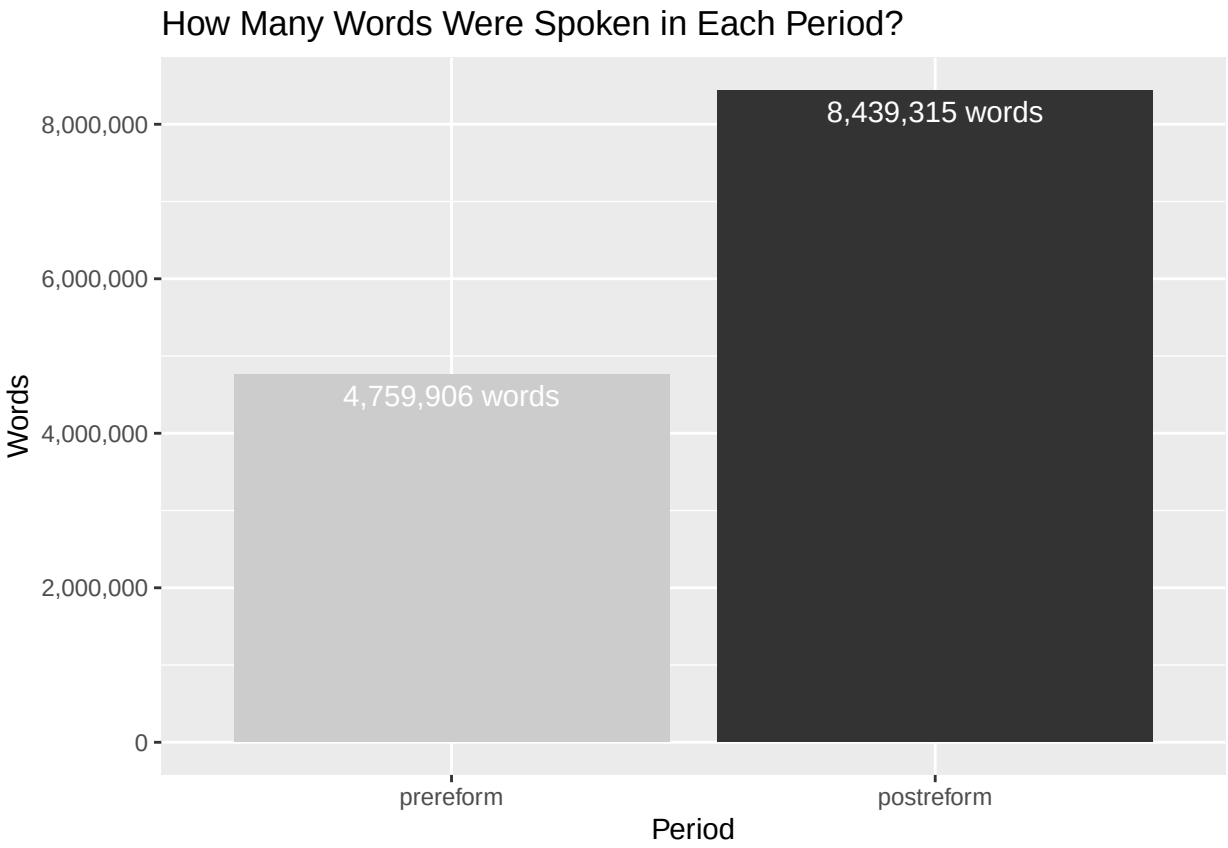
We can remove the variables that we are no longer using to save space ———

First, let's organize the data into two groupings – the words spoken prereform and those spoken postreform.

Let's break the data up into individual words and count how many words are in each category.

```
## Joining with `by = join_by(word)`
## Joining with `by = join_by(word)`
```

```
# Visualize the total number of words spoken in each period
# - geom_col() creates a bar chart
# - scale_x_discrete() orders the periods on the x-axis
# - scale_fill_grey() uses grayscale shading
# - scale_y_continuous() formats the y-axis with commas
# - geom_text() labels each bar with its total word count
# - labs() adds the title and axis labels
ggplot(both_periods_word_count_totals,
       aes(x = period, y = total, fill = period)) + # Plot periods on the x-axis, word totals on the y-
  geom_col(show.legend = FALSE) + # Create a bar chart
  scale_x_discrete(limits = c("prereform", "postreform")) + # Order the periods on the x-axis
  scale_fill_grey() + # Use grayscale shading
  scale_y_continuous(labels = comma) + # Format the y-axis with commas
  geom_text(aes(label = paste0(comma(total), " words")), # Label each bar
            color = "white", # Make the text white
            vjust = 1.5) + # # Vertically position the text inside the bar
  labs(title = "How Many Words Were Spoken in Each Period?", # Add a chart title
       x = "Period", # Label the x-axis
       y = "Words") # Label the y-axis
```



Many more words were spoken in the ten years after the reform than the ten years before. How many more? Text mining means we never have to guess.

```
## [1] 3679409
```

Next, let's look at the actual words spoken in each period and see how they compare.

Table 6.1: The Top Ten Words Spoken in Prereform and Postreform Parliament

period	word	wordcount
postreform	house	121,241
postreform	hon	119,156
postreform	noble	79,627
postreform	bill	66,044
postreform	lord	65,073
postreform	country	55,848
postreform	government	54,333
postreform	question	41,455

period	word	wordcount
postreform	ireland	40,072
postreform	subject	38,767
prereform	house	59,045
prereform	hon	58,821
prereform	country	38,776
prereform	noble	34,335
prereform	bill	32,251
prereform	government	27,383
prereform	lord	25,402
prereform	time	21,347
prereform	question	20,757
prereform	subject	20,726

The analyst will discern that there are only very minor differences between the top words of both categories. What are we to make of that?

We should begin by admitting that these words tell us little that's new about parliament or history. The word "house" was frequently used in parliament to refer to the House of Commons and House of Lords, the two chambers of parliament. Members of the houses traditionally addressed each other as "honorable sir" (abbreviated "hon" in Hansard) and (when addressing peers of the realm) "noble sir," "noble sirs," or "my lord." The general matters of parliament concerned the state of the "country," the passing of "bills" into law, the activities of the "government," various other "subjects," and the official matter of "questions" – a formal category of address about the business of parliament. To find the top words offers no real insight to any analyst who understands the basic business of parliament.

The only apparent difference in this table occurs in the words "time" – a top-ten word prereform but not postreform – and "Ireland" – a top-ten word postreform and prereform. If we looked at more than the top-ten words, even these differences would likely appear slight; the two words missing in this table are almost surely in the top twenty list for both periods – something we can easily check by adjusting the command `slice_head(n=10)` from 10 to 20.

In short, this step provides us with what data scientists refer to as a "validation" – that is, we have confirmed that the method of counting words tells us that the speech of parliament concerns the business of parliament.

A validation step is crucial to knowing that we are on the right track. If our attempt at counting (or any other statistical analysis) suggested that parliamentary speech in general concerned squids, whales, and cephalopods, or even obscure names of towns in Ireland, we would guess that something had gone wrong. Throughout the process of data analysis, we should use basic statistical measures as validation that the general idea is working. If our analysis tells us anything that doesn't make sense, we should rush to check our method – not revisit the laws of history.

0.0.2 Strategies for Reaching Insight

How do we move from validation to insight? There are many strategies. Let's think through a few:

- We can read a longer list of top words for each period, for instance the top 500 instead of the top 10. Keeping track of minute changes may tell us something. On the other hand, there will be a great many

words to look at. If the differences are slight (like a #9 vs a #12 position for a certain word from one period to the next) we risk losing track of which words are important for which periods.
- We can look at other aspects of the dataset, for instance the speaker column, asking how many words were spoken by each speaker in the prereform and postreform periods.
- We can use a variety of statistical methods to aggregate information about the speakers that were the most different and the words whose expression was most different in the prereform and postreform periods.

In the sections that follow, we will undertake each of these steps in turn. They will give us different answers.

Step #1 is easily accomplished by a quick adjustment in code, tweaking the command `slice_head(n=10)` from 10 to 500. Please change the command for youself and inspect the results. Is it easy to interpret them?

Step #2 is easily accomplished by revisiting the code in greater depth. We won't need any new commands, but we will need to apply the functions we've already seen – `group_by()`, `summarize()`, and `n()` – to the `speaker` field rather than the `year` field.

Let's do that now to answer the question, how many words did each speaker speak in the prereform and postreform period?

0.1 Investigating Speakers' Wordcounts, Pre- and Post-Reform

```
## # A tibble: 6 x 4
##   period     suggested_speaker   year wordcount
##   <chr>      <chr>              <dbl>     <int>
## 1 postreform aaron_chapman_3235  1834       382
## 2 postreform aaron_chapman_3235  1835       600
## 3 postreform aaron_chapman_3235  1836       477
## 4 postreform aaron_chapman_3235  1837       227
## 5 postreform aaron_chapman_3235  1838       455
## 6 postreform aaron_chapman_3235  1839        59
```

Let's look at the speakers who spoke the most.

```
## # A tibble: 5 x 2
##   suggested_speaker   wordsperspeaker
##   <chr>                         <int>
## 1 <NA>                       10243578
## 2 robert_peel_1664            2087481
## 3 joseph_hume_1712            1336979
## 4 john_russell_1885           1046022
## 5 henry_brougham_1679          858745
```

0.2 Investigating Speaker Wordcount With Joins

With the data frame both_periods_speakers, we have wordcounts for each speaker for pre- and postreform. There's a great deal we can do with this data. First, let's perform some basic counts to help us to understand, in raw numbers, the basic features of prereform and postreform parliament.

We'll start by asking how many members of parliament stood for the ten years pre and post reform. We will label each with a "class" called "prereform only," "postreform only," or "continuous speakers." Note that these "classes" are fundamentally different than the "period" categorization we applied above, where we labeled speeches given before the 1832 reform law as "prereform" and those after the law as "postreform." Instead of labeling the years, we want to identify the speakers that spoke only in one period or the other, and those who spoke in both.

To identify these speakers, we will begin by returning to an earlier data frame – `prereform_speakers_wordcount` – and cleaning it to exclude those speakers who also appear in the dataset `postreform_speakers_wordcount`.

Information scientists theorize several ways to combine data through what is called "set theory." Two of the simplest joins are known as the "union" of two sets – what they have in common – and the "difference" between two sets – what is in one but not the other.

To find the prereform-only and postreform-only speakers, we will find the "difference" between the two sets by using a function named `anti_join()`. The `anti_join()` function excludes from one dataset all the data that appear in a second dataset. If we begin with a dataset of all members of parliament who spoke prereform and perform an "anti-join" with the set of speakers who spoke postreform, the resulting dataset is a list of speakers who only spoke prereform.

```
# Find the speakers who appear only in the prereform period
# - anti_join() keeps speakers who do not appear in the postreform period
# - mutate() labels the speakers by category
prereform_only_speakers <- prereform_speakers %>%
  anti_join(postreform_speakers, by = "suggested_speaker") %>% # Keep speakers unique to the prereform p
  mutate(class = "prereform only") # Label the speaker category

# Display a random sample of prereform-only speakers
# - sample_n() selects 10 random speakers
prereform_only_speakers %>%
  sample_n(10) # Display 10 random speakers
```

```
## # A tibble: 10 x 2
##    suggested_speaker           class
##    <chr>                       <chr>
##  1 granville_ryder_2625        prereform only
##  2 edward_lloyd_1413           prereform only
##  3 arthur_duncombe_2651        prereform only
##  4 john_grossett_2204          prereform only
##  5 william_gordon-cumming_2770 prereform only
##  6 robert_montagu_4643         prereform only
##  7 john_wharton_636            prereform only
##  8 charles_abney-hastings_2449 prereform only
##  9 george_holme-sumner_1419    prereform only
## 10 george_sackville-west_6565  prereform only
```

How many speakers are there in our dataset? To answer the question, we just need to know the number of rows, given that the dataset lists one distinct speaker for each row. The function `nrow()` will provide an answer to how many rows there are in any dataset.

```
## [1] 255
```

Performing the same operation again, beginning with postreform speeches and removing the rows of data that appear prereform, yields a dataset of speakers who only spoke after the reform.

```
## # A tibble: 6 x 2
##   suggested_speaker       class
##   <chr>                   <chr>
## 1 aaron_chapman_3235      postreform only
## 2 adolphus_hanover_6515   postreform only
## 3 alex_bannerman_2832     postreform only
## 4 alexander_chisholm_3349 postreform only
## 5 alexander_hope_3700     postreform only
## 6 alexander_johnston_3683 postreform only
```

How many speakers were there postreform?

```
## [1] 502
```

We now have two sets of numbers – the count of unique speakers who spoke 1822-32 and 1832-42. The sets of numbers are new to history; there has never before been a count of these speakers. We can learn something from them. Before text mining, it was impossible to get a real impression of how many speakers were active in parliament before and after 1832. But the simple ability to count allows us to add detail to our accounts of the reform and its mechanisms.

The basic finding is that there were more speakers after the reform period. Our observation makes sense, because the reforms of 1832 introduced more popular consent into the democratic process. No longer were members appointed in "pocket buroughs" by aristocrats who held the sole authority for electing the representative. Now, all seats were open to a popular vote (albeit of those rich enough to pay at least £10 per year in taxes – no mean sum). If we know that the number of seats in the House of Commons were set at 658 before and after the reform (which we learn by consulting a secondary source, XXXXXX), the numbers tell us that this reform introduced more variation and turnover in who occupied the seats of parliament.

Substantively more different members of parliament were elected in the ten years following 1832 than in the decade before. How many more members were there among the postreform only members than the prereform only members?

```
## [1] 247
```

We can also investigate speakers from a third category – those who were continuously in parliament, who held office before the reform and who were able to secure election after the reform as well. How many such speakers were there?

Please note that we do not have the data to ask which members of parliament who held office before 1832 were elected to office after 1832. We do not have the data to support that question. What we have in Hansard is only a record of who was recorded as having spoken. This is a rather more discrete query, because not every elected member actually speaks when they are in office, and some of those who spoke might have spoken so little that the clerks recording speeches took no notice of what they said.

Because recorded parliamentary speech is an index of importance, the members who were recorded as having spoken before and after 1832 offers a list of those members who were important enough to have taken notice of the public in both periods. The overlap of members who appear in both the pre- and post-reform records is a good proxy for which members mattered in the public imagination and met the criterion for having been elected both before and after the reform.Do we imagine this number to have been large or small?

We might guess that it was vanishingly small, because a reform of historical significance could have swept away all incumbents. On the other hand, many historians of the 1832 Reform Bill have pointed out that it didn't shift the election process so drastically as had once been assumed.

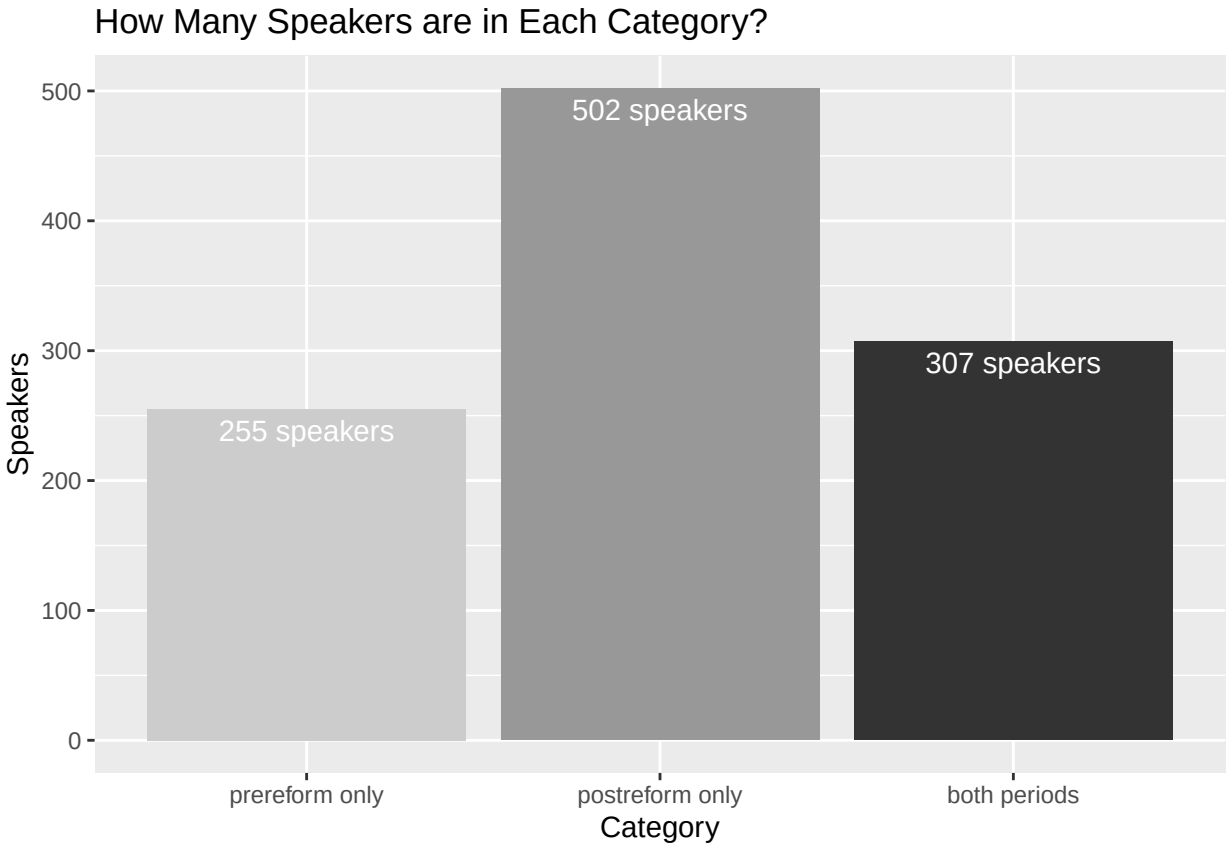
In any case, to identify the speakers who appear in both the prereform and postreform datasets, we will use another function – `inner_join()` – to identify what is known as the "intersection" of the two datasets. That is, we are looking for speakers who occupy a row in both the `prereform_speakers_wordcount` data frame and the `postreform_speakers_wordcount` data frame. The syntax for `inner_join()` is just like that of `anti_join()`, but the result is to keep rows that appear in both datasets, not to exclude those rows.

```
## [1] 307
```

We now have actual numbers for the members of parliament recorded as only having spoken before the reform, those who were recorded as only having spoken after the reform, and those who were recorded as having spoken both before and after the reform. Let's represent our results visually.

Rename all_classes and mention that we are going to use it a lot

```
# Visualize the number of speakers in each category
# - geom_col() creates a bar chart
# - scale_x_discrete() places the categories in the desired order
# - scale_fill_grey() uses grayscale shading
# - geom_text() labels each bar with the speaker count
# - labs() adds the title and axis labels
ggplot(all_classes_count,
       aes(x = class, y = speakercount, fill = class)) + # Plot categories, speaker counts, and bar fil
  geom_col(show.legend = FALSE) + # Create a bar chart
  scale_x_discrete(limits = c("prereform only", "postreform only", "both periods")) + # Place the categ
  scale_fill_grey() + # Use grayscale shading
  geom_text(aes(label = paste(speakercount, "speakers")), # Label each bar
            color = "white", # Make the text white
            vjust = 1.5) + # Position the text inside the bar
  labs(title = "How Many Speakers are in Each Category?", # Add a chart title
       x = "Category", # Label the x-axis
       y = "Speakers") # Label the y-axis
```



Certain caveats are important for interpreting this number. While the speaker data gives us reliable insights into relative trends in membership and speech, the numbers are only so good as our dataset.

One of the great advantages of text mining is that we can analyze not just the speakers in parliament, but the consequences of the 1832 Reform Act for what was spoken about in parliament. Let's begin by asking what parliament spoke about before and after the reform.

0.3 Working With Word Count

```
## # A tibble: 6 x 5
##   period     suggested_speaker   year wordcount class
##   <chr>      <chr>              <dbl>     <int> <chr>
## 1 postreform aaron_chapman_3235  1834       382 postreform only
## 2 postreform aaron_chapman_3235  1835       600 postreform only
## 3 postreform aaron_chapman_3235  1836       477 postreform only
## 4 postreform aaron_chapman_3235  1837       227 postreform only
## 5 postreform aaron_chapman_3235  1838       455 postreform only
## 6 postreform aaron_chapman_3235  1839        59 postreform only
```

Note that we have created two columns that annotate the speeches and speakers in different ways. The

"period" column refers to the year in question – were the n words said in 1831 prereform or postreform? The "class" column sorts the speakers themselves – did this speaker only speak in the prereform years, the postreform years, or in both periods?

We also have a wordcount column that reflects the words each speaker spoke per year.

Next, let's sort the data to find the speakers who spoke the most per year within each category – prereform, postreform, and continuously through the periods.

To find the speakers who spoke the most, we'll want to first create a new column, `wordcount_per_speaker`, which adds together each speaker's total wordcount. If we have multiple "wordcount" columns, we should add information about what each refers to. To make it less confusing, we'll rename our previous wordcount column `wordcount_per_year`. It's good practice to keep renaming fields in our datasets for clarity so that we don't get confused about what they mean. To calculate `wordcount_per_speaker`, we'll use the `group_by()` and `mutate()` functions that we've seen before. Note that we won't use `summarize()` in this case because we still want to keep the `wordcount_per_year()` field upon which the new count is based.

we'll use `group_by()` to group the data by "class," then use the `arrange()` and `slice_head()` functions to take the top wordcounts for each category.

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by class and suggested_speaker.
## i Output is grouped by class.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(class, suggested_speaker))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

```
## # A tibble: 6 x 4
## # Groups:   class [2]
##   class           suggested_speaker   wordcount_per_speaker top
##   <chr>           <chr>                             <int> <lgl>
## 1 both periods    <NA>                           10243578 TRUE
## 2 both periods    robert_peel_1664                2087481 TRUE
## 3 both periods    joseph_hume_1712                1336979 TRUE
## 4 both periods    john_russell_1885               1046022 TRUE
## 5 both periods    henry_brougham_1679              858745 TRUE
## 6 postreform only john_roebuck_2850                241977 TRUE
```

0.4 Annotating Our Data

Before we go any further, let's do some work to make the speaker names more elegant.

As the analyst will recall, because of the structure of our data, names that were algorithmically cleaned are labeled with a standardized name and number, e.g. "robert_peel_1665."

When we make a graph to share, we'd like to substitute Robert Peel's name for this awkward string. Therefore, we need to create a "key" where we can match the individual id "robert_peel_1665" to the name "Sir Robert Peel."

The names as given in the text are in the Hansard dataset under the `speaker` column. We can grab any one of them – minus the titles like "Lord Chancellor" with which some speakers are referenced – and it will

read like a normal name. The `str_detect()` function is useful for searching for a string. `!str_detect()` will allow us to throw out names that include titles we don't want to use. The `sample_n()` function is useful here as it tells the computer to select a random pick from a set; in other words, if we've grouped Robert Peel's many possible names, and we have the set "Robert Peel," "Sir Robert Peel," and "Mr. Robert Peel," the computer is instructed to pull out one of the names at random.

After we choose one name to use from the full Hansard list of literal speaker references, we can use these names to annotate the previous dataset, `all_speakers_annotated`, which has `wordcount` (per speaker per year) and `speaker`.

Let's try it.

```
## # A tibble: 5 x 1
##   suggested_speaker
##   <chr>
## 1 william_peel_1975
## 2 robert_peel_1664
## 3 edmund_peel_2792
## 4 jonathan_peel_2471
## 5 frederick_peel_4084
```

The results may not be impressive for the vast majority of speakers who spoke very little. But for major speakers like Robert Peel, the `speaker_key` dataset will help us reconcile the many names with which they appeared in the text with one numeric id number, then reconciling that numeric id with one recognizable name from the Hansard transcription.

1 Using Left Join to Annotate the Data

We've previously used set theory and "joins" to find the intersection and difference between two datasets. Here, we'll use another kind of join – the "union" – to annotate one dataset with information from another dataset.

That is, in `all_speakers_annotated`, we have a list of speakers and how many words they spoke every year. If we use `left_join()` to merge that dataset with the dataset `top_speakers()`, we'll have a record of whether each speaker was one of the most prolific speakers in their category. If we use `left_join()` to merge the dataset again with the dataset `speakers_key()`, we'll be able to use the literal transcriptions of speaker names to annotate the visualization later.

The function `left_join()` takes as its first argument the name of the dataset with which we're merging our data. A second optional argument is the "by" argument, which allows us to specify a column or set of columns to use for matching. In the case of the first join, we'll specify two fields, concatenated into a list, using the syntax `by = c("suggested_speaker", "class")`.

```
## Joining with `by = join_by(suggested_speaker)`
```

suggested_speaker	top	year	wordcount	class
edmund_peel_2792	NA	1832	71	both periods
robert_peel_1664	TRUE	1832	32476	both periods
robert_peel_1664	TRUE	1833	108710	both periods
robert_peel_1664	TRUE	1834	103120	both periods
robert_peel_1664	TRUE	1835	190727	both periods
robert_peel_1664	TRUE	1836	127812	both periods

2 Visualize the Results

Sometimes the most effective way of analyzing data is to "inspect" the results of counting, as we have done henceforth. For simple results, like a list of names or words, we prefer a table.

But for questions about information when several dimensions of data are related to each other – for instance speaker words over time when speakers have been categorized in three "classes" – visualization may be useful for conveying how many factors are related to each other. Next, we'll use `ggplot()` to create a visualization of the results as a timeline.

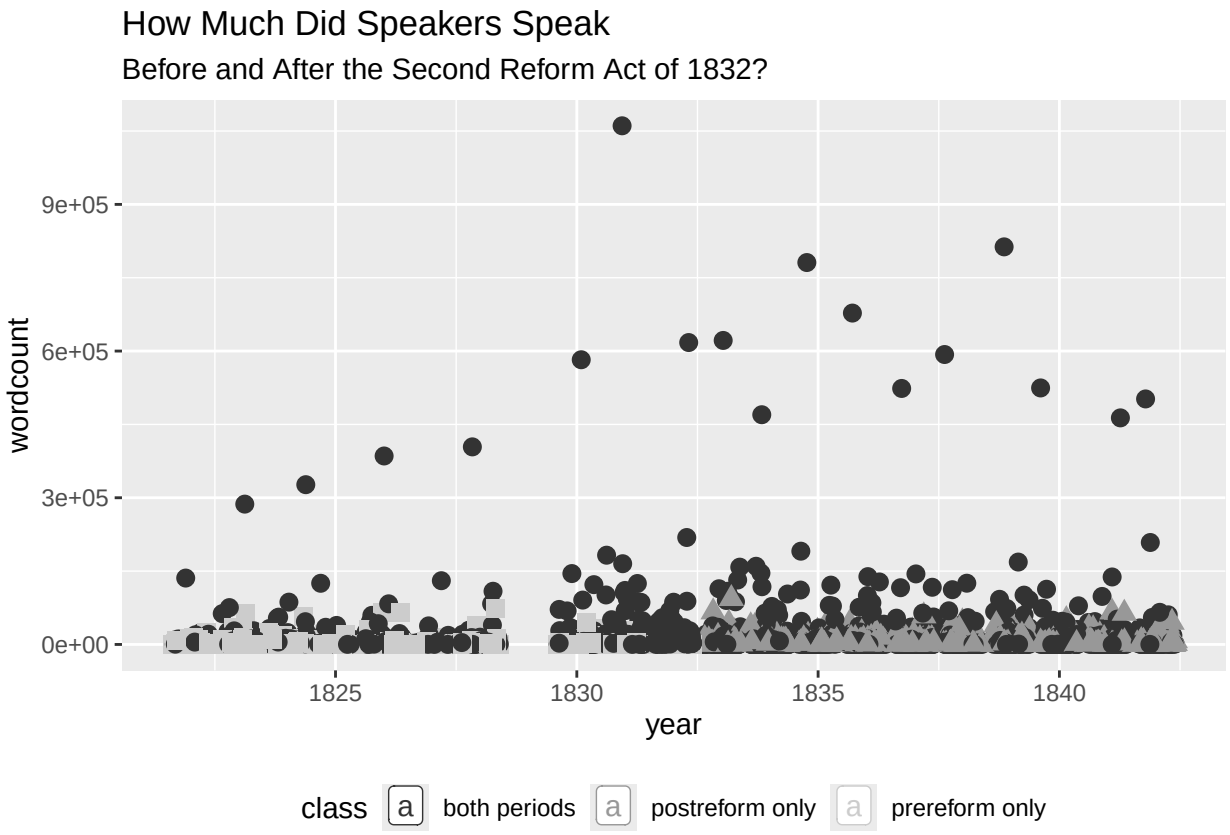
In the case of the data about speakers before and after 1832, we have rich information that deserves to be represented in an interesting way. We have three categories of speakers – prereform, postreform, and continuous speakers – and information about how many speakers are in each category. We also have information about how much each speaker spoke each year. Can we model the information together and learn something from the result?

in the code block below, we'll create a "dot plot," mapping one point for each of the 10,810 speakers in the data frame. We will use the color and shape of the dots to map the three classes of speakers – prereform, postreform, and continuous.

2.0.1 Contingent labeling

Because labeling all the speakers would result in an unreadable graphic, we will only label those speakers who are classified as `TRUE` in the column `top`, where we annotated if a speaker was among the top n speakers of each class by wordcount. The syntax for this operation is given inside the parentheses of `geom_label_repel()`, where the coder has an opportunity to point to a new dataset as the source of the text in the labels. Here, we tell the computer which dots to label and which to leave blank with the use of `subset(graph_ready, top == TRUE)`. Operations of this type are extremely useful for the kind of dense, text-forward information that implementors of text mining often want from their analysis.

```
## Warning: Removed 21 rows containing missing values or values outside the scale range
## (‘geom_label_repel()‘).
```



What do we learn from this plot? A great deal. One is about the relatively higher counts of speeches in the 1830s than the 1820s, irrespective of the reform Act. Speeches by Lord Althorp, Robert Peel, and Henry Brougham regularly reached into range of 150,000 to 200,000 words – which if delivered at breakneck speed would have taken a stunning fifteen hours to deliver.

Even the speeches of lesser speakers were longer in the new decade, stretching towards 50,000 words instead of 20,000 words as the upper limit for most rank-and-file speakers – meaning that after the reform, multiple speakers each year addressed the house for five hours or more at a time.

We also learn that by and large it was the continuous speakers – not the prereform-only aristocratic appointees or the postreform-only members elected by the middle class – who spoke at the greatest length. Represented with black dots, these men included prime ministers like Robert Peel.

We can also use this visualization to note the few prereform-only speakers who delivered relatively long speeches, for instance Mr. Canning and Mr. Huskisson, the latter important to the Board of Trade, represented by light gray squares.

Equally exceptional were the few members of parliament elected only after 1832 who spoke with prominence, among them John Roebuck and Frederick Shaw. Both men whose careers were endorsed by reform swam in a sea of seasoned representatives who had first joined parliament in an earlier era.

Note that our visualization is only as good as the quality of the data. Where speakers have not been reconciled correctly – as is the case with "Lord Brougham" (classified as postreform only), who in reality is

the same as "Henry Brougham" (classified as a continuous speaker) – they may appear as discrete speakers when indeed they are the same.

2.1 Using Distinctiveness to Investigate Language in the Period Before and After 1832

Interesting though it is to quantify the number of speakers and speeches before and after the Reform Act, far more consequential is the question of whether ideas or issues changed as new voters participated in the parliamentary process.

In general, historians have told us that after the 1832 Reform Act, Parliament began to pay more attention to issues that reflected the concerns of the growing middle class and urban population—concerns that had been largely ignored by the pre-reform, aristocracy-dominated Parliament. Key shifts included ending child labor, the repeal of the grain laws to make bread cheaper for urban workers, public health, tackling corruption, and grants for schools.

Yet the debates of parliament cover a diversity of issues far wider than the lexical set of changes that have been enshrined in history textbooks. If we really want to know, in full, how parliament sounded different before and after 1832, text mining provides one of our greatest possible tools for understanding.

Text mining gives us an opportunity to approach the issue on the basis of the words that speakers used. The tools of analyzing distinctiveness, introduced in the previous chapter, allow us to prioritize words that were used markedly more in one case or another. For example, we might become interested in which words were used the most before and after 1832.

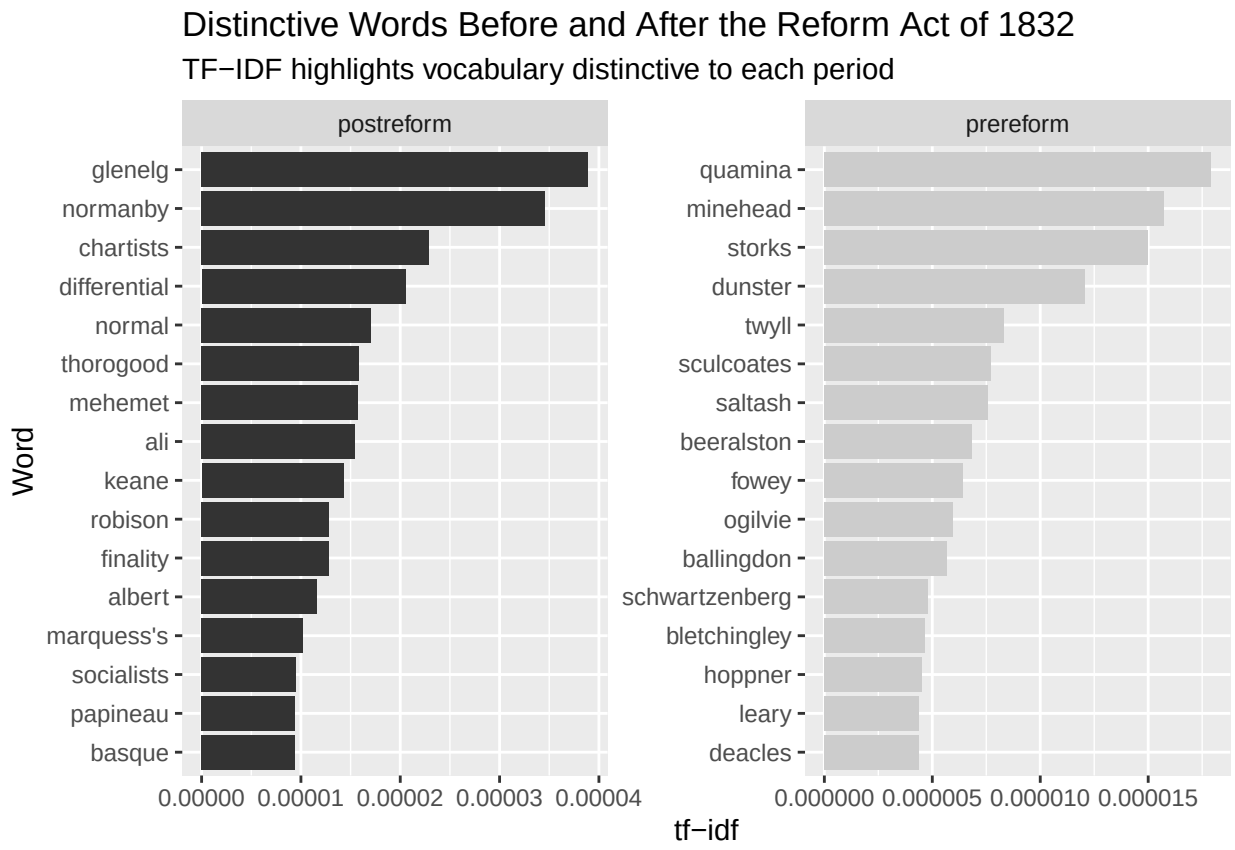
By using tf-idf as our tool for understanding, we will privilege a strict filter: we will look for words spoken in 1822-32 that were almost never spoken in 1832-42, and vice-versa. What might the silences and expansions of the period reveal that we did not previously know?

```
# Calculate the tf-idf score for each word in each period
# - bind_rows() combines the prereform and postreform word counts
# - bind_tf_idf() calculates how distinctive each word is within each period
period_tfidf <- bind_rows(prereform_word_count, postreform_word_count) %>%
  bind_tf_idf(word, period, wordcount) # Calculate the tf-idf scores

# Keep the most distinctive words from each period
# - group_by() groups the words by period
# - slice_max() keeps the 15 words with the highest tf-idf scores in each period
# - ungroup() removes the grouping
top_words <- period_tfidf %>%
  group_by(period) %>% # Group by period
  slice_max(order_by = tf_idf, n = 15) %>% # Keep the top 15 words in each period
  ungroup() # Remove the grouping

# Visualize the most distinctive words in each period
# - reorder_within() orders words by tf-idf separately within each period
# - geom_col() creates a bar chart
# - facet_wrap() creates one panel for each period
# - coord_flip() makes the word labels easier to read
```

```
# - scale_x_reordered() preserves the word order within each panel
# - scale_y_continuous() formats the tf-idf scores with commas
# - scale_fill_grey() uses grayscale shading
# - labs() adds the title, subtitle, and axis labels
ggplot(top_words,
       aes(x = reorder_within(word, tf_idf, period), # Order words by tf-idf separately within each per
           y = tf_idf, # Plot tf-idf scores on the y-axis
           fill = period)) + # Shade bars by period
  geom_col(show.legend = FALSE) + # Create a bar chart and hide the legend
  facet_wrap(~period, scales = "free") + # Create one panel for each period
  coord_flip() + # Flip the axes
  scale_x_reordered() + # Preserve the word order within each panel
  scale_y_continuous(labels = label_comma()) + # Format the tf-idf scores
  scale_fill_grey() + # Use grayscale shading
  labs(title = "Distinctive Words Before and After the Reform Act of 1832", # Add a chart title
       subtitle = "TF-IDF highlights vocabulary distinctive to each period", # Add a subtitle
       x = "Word", # Label the x-axis
       y = "tf-idf") # Label the y-axis
```



The results are interesting. The top most ranked prereform word is “quamina,” a word that references

Quamina Gladstone, the leader of the 1823 slave revolt in Demerara led by an enslaved carpenter born in Ghana and sold into slavery with his mother in 1817. Gladstone was prohibited from taking time off of work to look after his ill wife, Peggy. After Peggy's death, Quamina's son Jack Gladstone organized a peaceful strike of thousands. In the aftermath of the revolt, Quamina was assassinated, his body displayed in chains beside the public road.

Quamina Gladstone's story was important to prereform antislavery movements in Britain; Quamina's revolt was used to argue the case that the conditions of slavery were so miserable that they merited abolition. But By 1832-33, when slavery was under debate, Quamina's case was forgotten. The slave revolts still spoken about in 1833 were those in Haiti and Jamaica, associated with the murder of white slavers. White fear of violence was used as a pretext for extending slavery under the guise of "apprenticeships."

Other words too point at important differences between the prereform and postreform periods. Postreform, the terms "chartists" and "socialist" point to the movement to give the vote to the working class and experiments with government welfare associated at the time with the factory owner and utopian theorist Robert Owen. The word "normal" suggests statistics, but in context, parliamentary speakers of the 1830s typically invoked the word to refer to the new idea of founding colleges specifically for teachers, patterned after the famous Ecole Normale Superior in Paris.

Postreform terms such as "Glenelg" and "Normanby" reference individuals who played a major role in the expansion of empire. Charles Grant, 1st Baron Glenelg, served as Chief Secretary for Ireland after 1819 and President of the Board of Trade after 1828. He helped to author the new Government of India Act of 1833, which transfered power away from the East India Company.

A more complete research project would use this list of distinctive words to investigate further; elsewhere, we discuss how to navigate the debates.

2.1.1 Understanding Language Shifts With a More Generous Measure: Log Likelihood

Distinctiveness can be measured in multiple ways. When we compare oranges and apples, we may choose to only note the strictest differences (one has segments, one's flesh is continuous and dense), or we may make a more general list of overlapping trends that are slightly different (both have seeds, bugt one's is white and the other's is dark). Each time we change the basis of our comparison, we have the opportunity to illuminate something new about difference – the first time, emphasizing exclusion, the second time, emphasizing overlap.

As Prof. Guldi has explained in *The Dangerous Art of Text Mining*, different approaches to distinctiveness can be used to explore the multiple dimensions of temporal change. A skilled digital historian will refuse to settle for one set of answers about distinctiveness, instead choosing to explore many metrics.

If tf-idf is a tool for identifying the characterstics of an orange that can't be found in apples, another approach – log likelihood – helps us see the nuances that overlap. The mathematical features of this approach and their implications are explored in *The Dangerous Art of Text Mining*, so here, we will emphasize instead how to employ log likelihood to create an alternative comparison of two time periods.[3]

The `dhmeasures` package contains the `log_likelihood()` function, which acts much like `bind_tf_idf()`, accepting a data frame with columns `word`, `group`, and `n`, and producing a score of distinctiveness ("ll") for each word-group pair.

```
# Combine the word counts from both periods
# - bind_rows() combines the prereform and postreform word counts
# - rename() changes wordcount to n for the log-likelihood calculation
```

```r
all_word_counts <- bind_rows(prereform_word_count, postreform_word_count) %>%
  rename(n = wordcount) # Rename the word count column

# Calculate the log-likelihood score for each word
# - log_likelihood() compares word use between the two periods
# - transmute() keeps the word and renames the score column
ll_scores <- log_likelihood(all_word_counts,
                            group = "period", # Compare words by period
                            group_list = c("prereform", "postreform")) %>% # Compare periods
  transmute(word, log_likelihood = prereform) # Keep the word and log-likelihood score

# Keep the prereform word counts
# - filter() keeps only the prereform period
# - select() keeps the word and renames the count column
pre_counts <- all_word_counts %>%
  filter(period == "prereform") %>% # Keep the prereform period
  select(word, pre_reform = n) # Keep the word and prereform count

# Keep the postreform word counts
# - filter() keeps only the postreform period
# - select() keeps the word and renames the count column
post_counts <- all_word_counts %>%
  filter(period == "postreform") %>% # Keep the postreform period
  select(word, post_reform = n) # Keep the word and postreform count

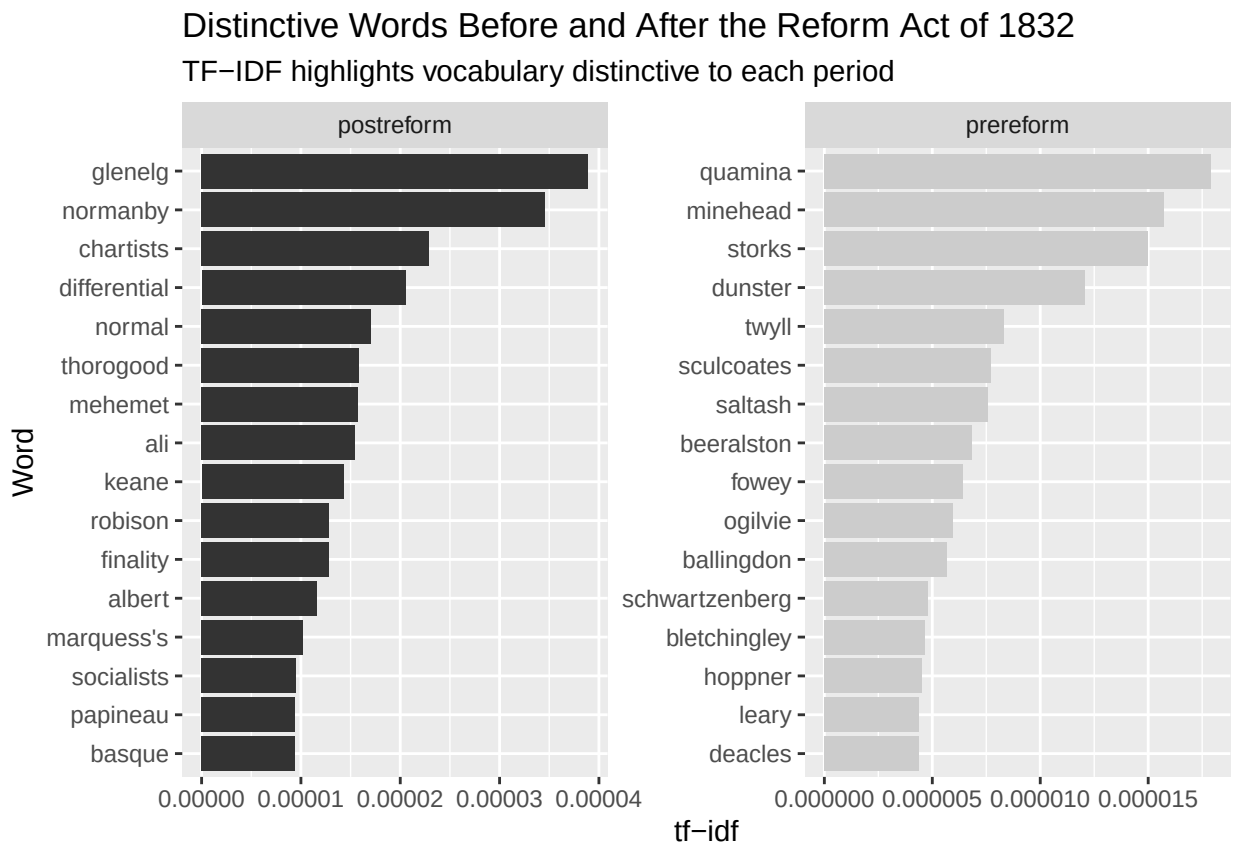
# Combine the word counts from both periods
# - full_join() keeps all words from both periods
# - coalesce() replaces missing counts with zero
word_counts <- full_join(pre_counts, post_counts, by = "word") %>%
  mutate(pre_reform = coalesce(pre_reform, 0L), # Replace missing prereform counts with zero
         post_reform = coalesce(post_reform, 0L)) # Replace missing postreform counts with zero

# Find the total number of words in each period
# - sum() adds the word counts within each period
pre_total <- sum(word_counts$pre_reform) # Find the prereform total
post_total <- sum(word_counts$post_reform) # Find the postreform total

# Find relative word frequencies and identify the more distinctive period
# - left_join() adds the log-likelihood scores
# - mutate() calculates relative frequencies and labels the more common period
distinctive_words <- word_counts %>%
  left_join(ll_scores, by = "word") %>% # Add the log-likelihood scores
  mutate(pre_rate = pre_reform / pre_total, # Find the prereform relative frequency
         post_rate = post_reform / post_total, # Find the postreform relative frequency
         distinctive_period = if_else(pre_rate > post_rate,
                                      "pre_reform",
                                      "post_reform")) # Label the period where the word is more common
```

```r
# Keep the most distinctive tf-idf words from each period
# - group_by() groups the words by period
# - slice_max() keeps the 15 words with the highest tf-idf scores
# - ungroup() removes the grouping
top_words <- period_tfidf %>%
  group_by(period) %>% # Group by period
  slice_max(order_by = tf_idf, n = 15) %>% # Keep the top 15 words in each period
  ungroup() # Remove the grouping
```

```r
# Visualize the most distinctive words in each period
# - reorder_within() orders words by tf-idf separately within each period
# - geom_col() creates a bar chart
# - facet_wrap() creates one panel for each period
# - coord_flip() makes the word labels easier to read
# - scale_x_reordered() preserves the word order within each panel
# - scale_y_continuous() formats the tf-idf scores with commas
# - scale_fill_grey() uses grayscale shading
# - labs() adds the title, subtitle, and axis labels
ggplot(top_words,
       aes(x = reorder_within(word, tf_idf, period), # Order words by tf-idf separately within each per
           y = tf_idf, # Plot tf-idf scores on the y-axis
           fill = period)) + # Shade bars by period
  geom_col(show.legend = FALSE) + # Create a bar chart and hide the legend
  facet_wrap(~period, scales = "free") + # Create one panel for each period
  coord_flip() + # Flip the axes
  scale_x_reordered() + # Preserve the word order within each panel
  scale_y_continuous(labels = label_comma()) + # Format the tf-idf scores
  scale_fill_grey() + # Use grayscale shading
  labs(title = "Distinctive Words Before and After the Reform Act of 1832", # Add a chart title
       subtitle = "TF-IDF highlights vocabulary distinctive to each period", # Add a subtitle
       x = "Word", # Label the x-axis
       y = "tf-idf") # Label the y-axis
```



2.2 Distinctiveness Continued: Those Who Weren't Reelected

The mathematics of distinctiveness can be applied to the same dataset in myriad ways. Instead of comparing words spoken in different times, our analysis might become concerned with the words spoken by different kinds of speakers. For instance, some speakers who held parliamentary seats before 1832 weren't reelected, died, or chose not to stand for election in the reformed parliament. We might conjecture that these speakers had different characteristics than the speakers who held their seats before and after the Reform Bill.

To be sure, the analysis would be better if we only measured the effects of change on members of the House of Commons, whose seats are elected, rather than the appointed members of the House of Lords. In one version of the data, we are able to pull apart these questions. But in the slightly more efficient dataset available in the hansardr package, we have neglected information about the house in which each speech originated.

```
# Count the words spoken by each speaker in the prereform period
# - count() counts the occurrences of each word spoken by each speaker
prereform_speakers_wordcount <- prereform_words %>%
  count(suggested_speaker, word, name = "wordcount") # Count words by speaker

# Do the same thing to count the words spoken by each speaker in the postreform period
```

```r
postreform_speakers_wordcount <- postreform_words %>%
  count(suggested_speaker, word, name = "wordcount") # Count words by speaker
```

```r
# Find the speakers who appear in the prereform period
# - distinct() keeps one row for each speaker
prereform_unique_speakers <- prereform_speakers_wordcount %>%
  distinct(suggested_speaker) # Keep unique speakers

# Do the same thing to find the speakers who appear in the postreform period
postreform_unique_speakers <- postreform_speakers_wordcount %>%
  distinct(suggested_speaker)
```

```r
# Find the speakers who appear in both periods
# - inner_join() keeps only speakers found in both periods
# - mutate() labels the speaker category
continuous_speakers <- prereform_unique_speakers %>%
  inner_join(postreform_unique_speakers, by = "suggested_speaker") %>% # Keep speakers appearing in bot
  mutate(class = "both periods") # Label the speaker category

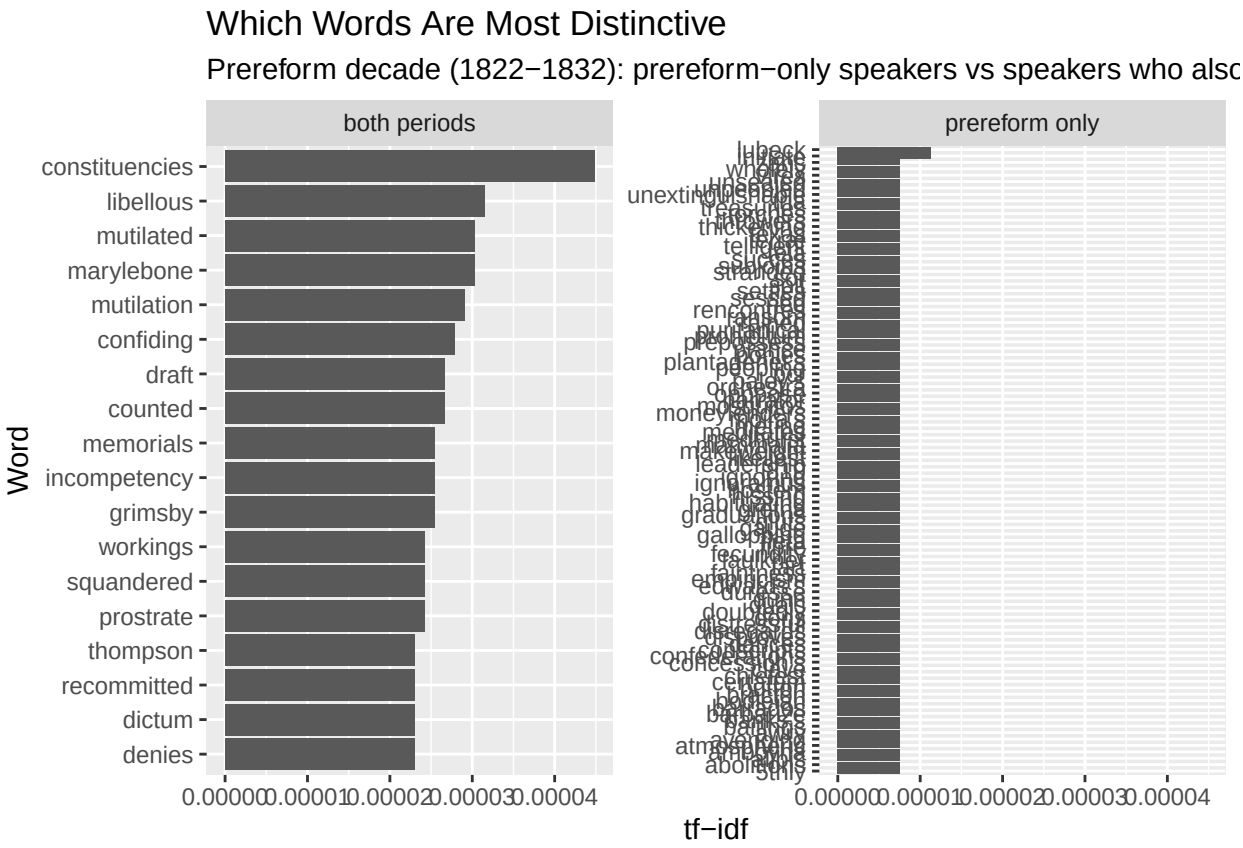
# Do the same to find speakers who appear only in the prereform period
prereform_only_speakers <- prereform_unique_speakers %>%
  anti_join(postreform_unique_speakers, by = "suggested_speaker") %>%
  mutate(class = "prereform only")

# Do the same to find the speakers who appear only in the postreform period
postreform_only_speakers <- postreform_unique_speakers %>%
  anti_join(prereform_unique_speakers, by = "suggested_speaker") %>%
  mutate(class = "postreform only")
```

```r
# Calculate tf-idf scores for speaker categories in the prereform period
# - left_join() adds the speaker category for each speaker
# - count() counts the words spoken by each speaker category
# - bind_tf_idf() calculates the tf-idf score for each word
prereform_words_w_speaker_classifications_tfidf <- prereform_speakers_wordcount %>%
  left_join(all_classes, by = "suggested_speaker") %>% # Add the speaker category
  count(class, word, name = "wordsperclass") %>% # Count words by speaker category
  bind_tf_idf(word, class, wordsperclass) # Calculate the tf-idf scores

# Keep the most distinctive words for each speaker category
# - group_by() groups the words by speaker category
# - slice_max() keeps the 15 words with the highest tf-idf scores
# - ungroup() removes the grouping
prereform_top_words_w_speaker_classifications_tfidf <- prereform_words_w_speaker_classifications_tfidf %
  group_by(class) %>% # Group by speaker category
  slice_max(order_by = tf_idf, n = 15) %>% # Keep the top 15 words
  ungroup() # Remove the grouping
```

```r
# Visualize the most distinctive words for each speaker category
# - reorder_within() orders words by tf-idf separately within each speaker category
# - geom_col() creates a bar chart
# - facet_wrap() creates one panel for each speaker category
# - coord_flip() makes the word labels easier to read
# - scale_x_reordered() preserves the word order within each panel
# - scale_y_continuous() formats the y-axis labels
# - labs() adds the title and axis labels
ggplot(prereform_top_words_w_speaker_classifications_tfidf,
       aes(x = reorder_within(word, tf_idf, class), # Order words by tf-idf separately within each spea
           y = tf_idf)) + # Plot tf-idf scores on the y-axis
  geom_col() + # Create a bar chart
  facet_wrap(~class, scales = "free_y") + # Create one panel per speaker category
  coord_flip() + # Flip the axes
  scale_x_reordered() + # Preserve the word order within each panel
  scale_y_continuous(labels = label_comma()) + # Format the y-axis labels
  labs(title = "Which Words Are Most Distinctive", # Add a chart title
       subtitle = "Prereform decade (1822-1832): prereform-only speakers vs speakers who also spoke aft
       y = "tf-idf", # Label the y-axis
       x = "Word") # Label the x-axis
```

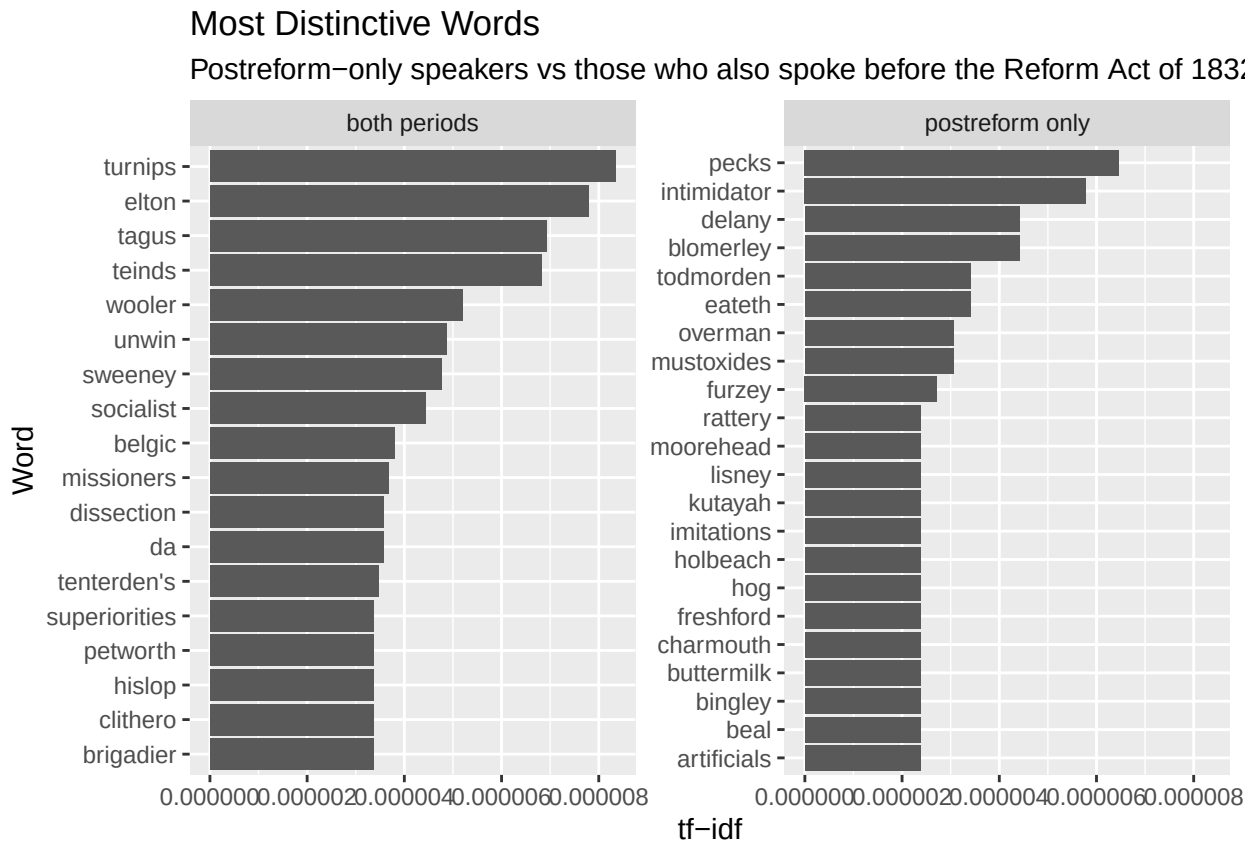


The same transformation can be applied to postreform-only speakers. Were the statesmen newly elected in 1832-33 different in their words from those who had held office continually?

```
# Calculate tf-idf scores for speaker categories in the postreform period
# - left_join() adds the speaker category for each speaker
# - count() counts the words spoken by each speaker category
# - bind_tf_idf() calculates the tf-idf score for each word
postreform_words_w_speaker_classifications_tfidf <- postreform_words %>%
  left_join(all_classes, by = "suggested_speaker") %>% # Add the speaker category
  count(class, word, name = "words_per_period") %>% # Count words by speaker category
  bind_tf_idf(word, class, words_per_period) # Calculate the tf-idf scores

# Keep the most distinctive words for each speaker category
# - group_by() groups the words by speaker category
# - slice_max() keeps the 15 words with the highest tf-idf scores
# - ungroup() removes the grouping
postreform_top_words_w_speaker_classifications_tfidf <- postreform_words_w_speaker_classifications_tfidf
  group_by(class) %>% # Group by speaker category
  slice_max(order_by = tf_idf, n = 15) %>% # Keep the top 15 words
  ungroup() # Remove the grouping
```

```r
# Visualize the most distinctive words for each speaker category
# - reorder_within() orders words by tf-idf separately within each speaker category
# - geom_col() creates a bar chart
# - facet_wrap() creates one panel for each speaker category
# - coord_flip() makes the word labels easier to read
# - scale_x_reordered() preserves the word order within each panel
# - scale_y_continuous() formats the y-axis labels
# - labs() adds the title and axis labels
ggplot(postreform_top_words_w_speaker_classifications_tfidf,
       aes(x = reorder_within(word, tf_idf, class), # Order words by tf-idf separately within each spea
           y = tf_idf)) + # Plot tf-idf scores on the y-axis
  geom_col() + # Create a bar chart
  facet_wrap(~class, scales = "free_y") + # Create one panel per speaker category
  coord_flip() + # Flip the axes
  scale_x_reordered() + # Preserve the word order within each panel
  scale_y_continuous(labels = label_comma()) + # Format the y-axis labels
  labs(title = "Most Distinctive Words", # Add a chart title
       subtitle = "Postreform-only speakers vs those who also spoke before the Reform Act of 1832", # A
       y = "tf-idf", # Label the y-axis
       x = "Word") # Label the x-axis
```



Again, a list of words emerges which is hard to interpret. We choose to ask the analyst to supply their own interpretation in the exercises for this chapter.

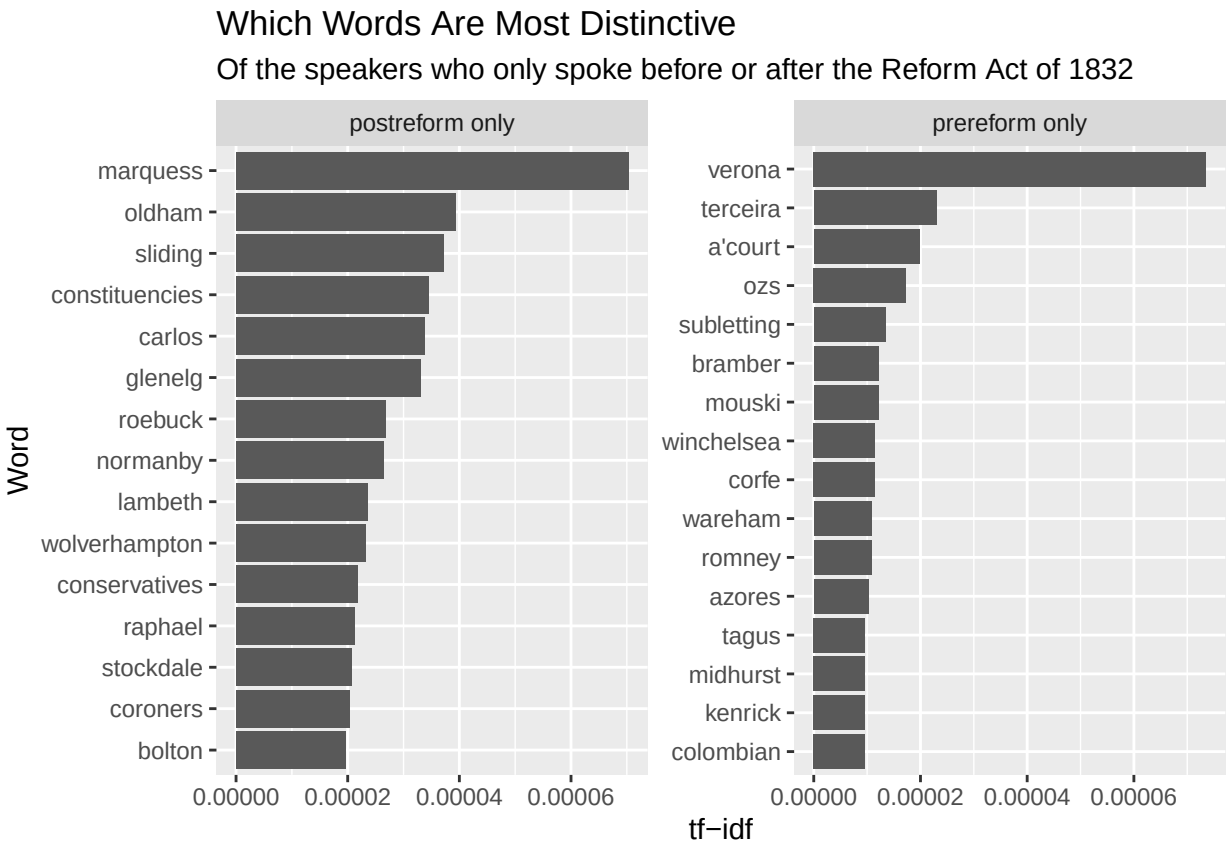
An alternative approach to measurement is to look exclusively at the speakers who only held office before or after the Reform Act. This means measuring the words of those who weren't reelected after 1832 with those who were only elected in 1832 or after.

```
# Calculate tf-idf scores for prereform-only and postreform-only speakers
# - bind_rows() combines the prereform and postreform words
# - left_join() adds each speaker's category
words_w_speaker_classifications <- prereform_words %>%
  bind_rows(postreform_words) %>% # Combine the two periods
  left_join(all_classes, by = "suggested_speaker") # Add the speaker category

# Calculate tf-idf scores for each speaker category
# - count() counts the words spoken by each category
# - bind_tf_idf() finds the tf-idf score for each word
words_w_speaker_classifications_tfidf <- words_w_speaker_classifications %>%
  count(class, word, name = "words_per_period") %>% # Count words by speaker category
  bind_tf_idf(word, class, words_per_period) # Find tf-idf scores
```

```r
# Keep the most distinctive words for prereform-only and postreform-only speakers
# - filter() removes speakers who appear in both periods
# - group_by() groups the words by speaker category
# - slice_max() keeps the 15 words with the highest tf-idf scores
# - ungroup() removes the grouping
top_words_w_speaker_classifications_tfidf <- words_w_speaker_classifications_tfidf %>%
  filter(class != "both periods") %>% # Remove speakers who appear in both periods
  group_by(class) %>% # Group by speaker category
  slice_max(order_by = tf_idf, n = 15) %>% # Keep the top 15 words
  ungroup() # Remove the grouping
```

```r
# Visualize the most distinctive words for prereform-only and postreform-only speakers
# - reorder_within() orders words by tf-idf separately within each speaker category
# - geom_col() creates a bar chart
# - facet_wrap() creates one panel for each speaker category
# - coord_flip() makes the word labels easier to read
# - scale_x_reordered() preserves the word order within each panel
# - scale_y_continuous() formats the y-axis labels
# - labs() adds the title and axis labels
ggplot(top_words_w_speaker_classifications_tfidf,
       aes(x = reorder_within(word, tf_idf, class), # Order words by tf-idf separately within each spea
           y = tf_idf)) + # Plot tf-idf scores on the y-axis
  geom_col() + # Create a bar chart
  facet_wrap(~class, scales = "free") + # Create one panel per speaker category
  coord_flip() + # Flip the axes
  scale_x_reordered() + # Preserve the word order within each panel
  scale_y_continuous(labels = label_comma()) + # Format the y-axis labels
  labs(title = "Which Words Are Most Distinctive", # Add a chart title
       subtitle = "Of the speakers who only spoke before or after the Reform Act of 1832", # Add a subt
       x = "Word", # Label the x-axis
       y = "tf-idf") # Label the y-axis
```



2.3 Critical Thinking About the Results

Parliament included many backbench MPs whose total output in words was relatively scarce. Does it matter if we only look at the most prominent speakers?

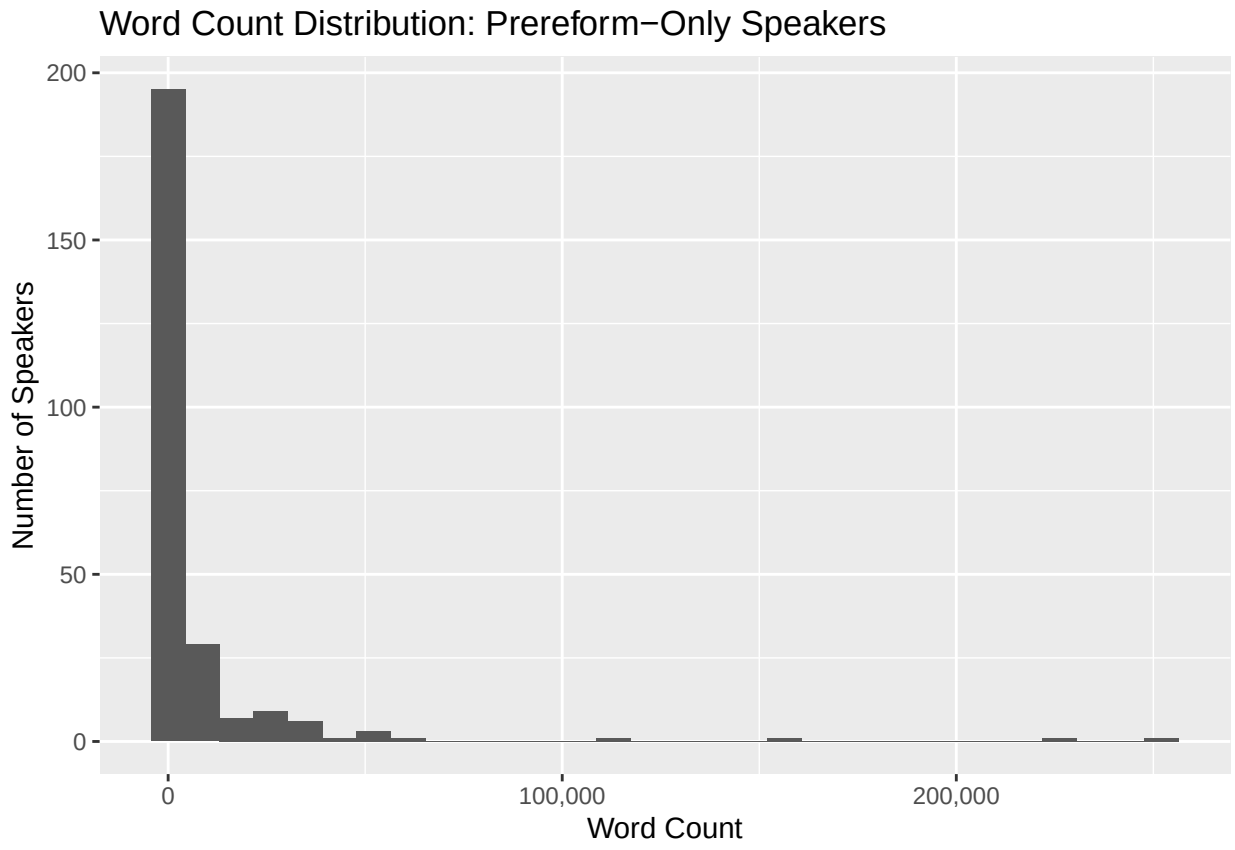
This kind of thinking can help us to refine our approach to research. The first step is to investigate how many speakers spoke in each period, and how much each contributed.

```
# Calculate the total words spoken by prereform-only speakers
# - filter() keeps only prereform-only speakers
# - group_by() groups the data by speaker
# - summarize() adds the total words spoken by each speaker
prereform_only_speakers_wordcount <- all_speakers_annotated %>%
  filter(class == "prereform only") %>% # Keep prereform-only speakers
  group_by(suggested_speaker) %>% # Group by speaker
  summarize(wordcount = sum(wordcount)) # Find the total words spoken

# Visualize the distribution of speaker word counts
# - geom_histogram() groups speakers into word-count bins
```

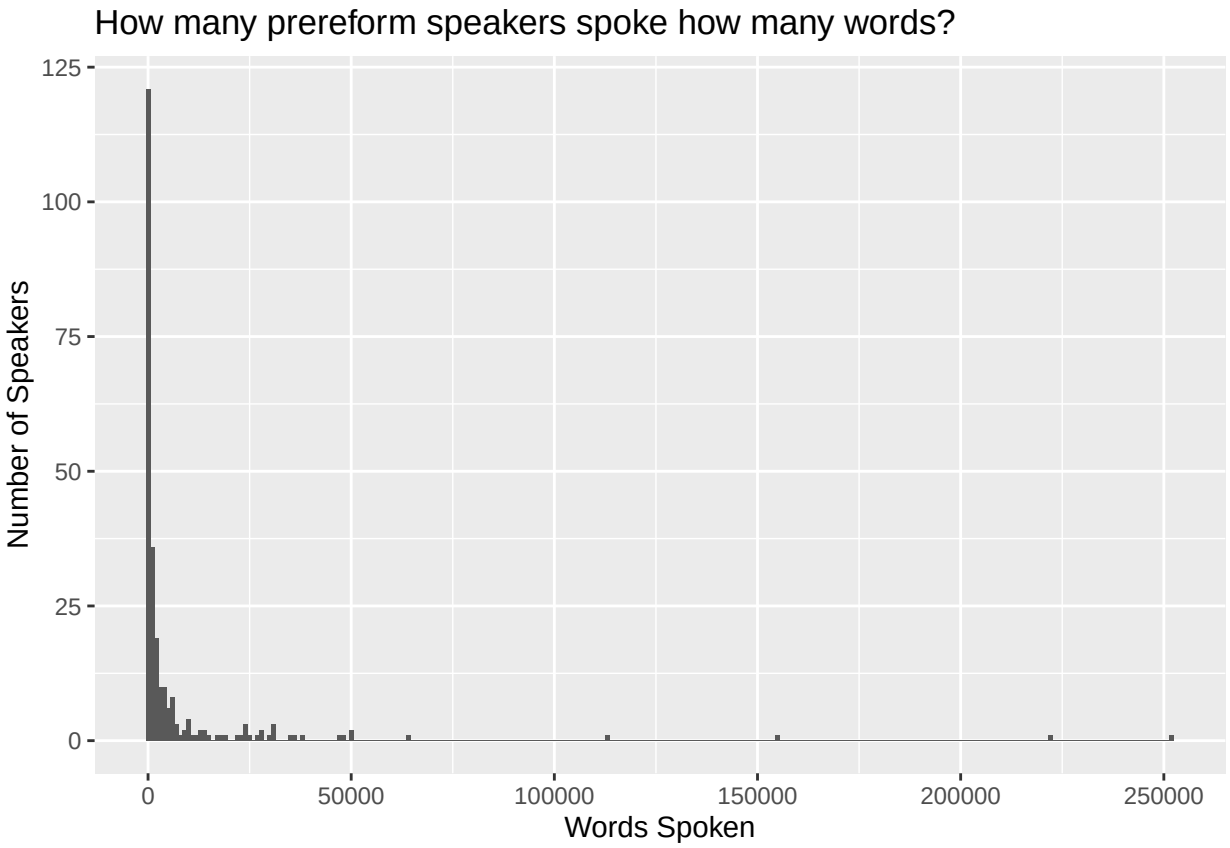
```
# - scale_x_continuous() formats the x-axis labels with commas
# - scale_y_continuous() formats the y-axis labels with commas
# - labs() adds the title and axis labels
ggplot(prereform_only_speakers_wordcount,
       aes(x = wordcount)) + # Plot word counts on the x-axis
  geom_histogram() + # Create a histogram
  scale_x_continuous(labels = comma) + # Format the x-axis labels
  scale_y_continuous(labels = comma) + # Format the y-axis labels
  labs(title = "Word Count Distribution: Prereform-Only Speakers", # Add a chart title
       x = "Word Count", # Label the x-axis
       y = "Number of Speakers") # Label the y-axis
```

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```




```
# Visualize the distribution of speaker word counts
# - geom_histogram() groups speakers into 1,000 word bins
# - labs() adds the title and axis labels
ggplot(prereform_only_speakers_wordcount,
```

```
      aes(x = wordcount)) + # Plot word counts on the x-axis
  geom_histogram(binwidth = 1000) + # Create 1,000 word bins
  labs(title = "How many prereform speakers spoke how many words?", # Add a chart title
       x = "Words Spoken", # Label the x-axis
       y = "Number of Speakers") # Label the y-axis
```

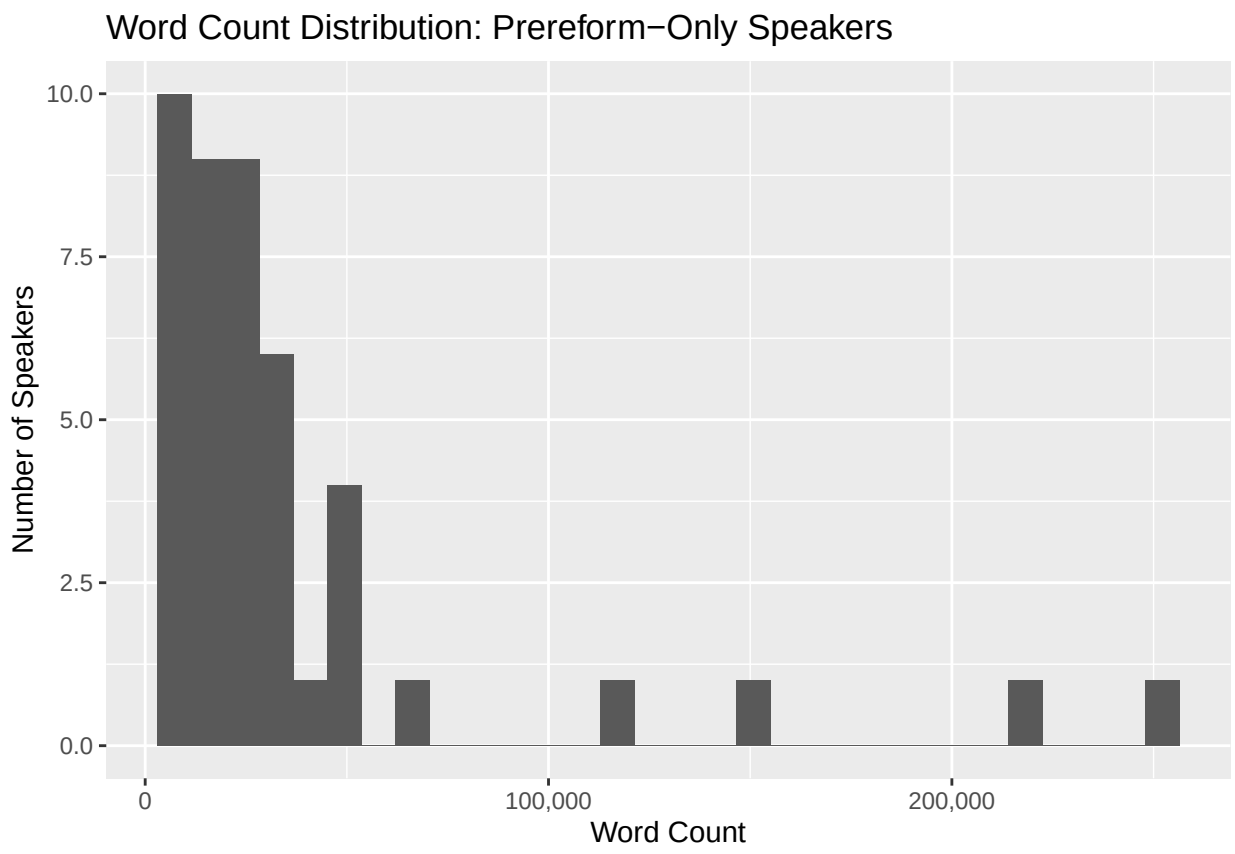


Many speakers – nearly 2000 in all – only spoke less than 7000 words, total, in the prereform period. Let's look at just those who spoke 7000 words or more.

```
# Keep prereform-only speakers who spoke at least 7,000 words
# - filter() keeps only prereform-only speakers
# - group_by() groups the data by speaker
# - summarize() adds the total words spoken by each speaker
# - filter() keeps speakers with at least 7,000 words
prereform_only_speakers_wordcount <- all_speakers_annotated %>%
  filter(class == "prereform only") %>% # Keep prereform-only speakers
  group_by(suggested_speaker) %>% # Group by speaker
  summarize(wordcount = sum(wordcount)) %>% # Find the total words spoken
  filter(wordcount >= 7000) # Keep speakers with at least 7,000 words
```

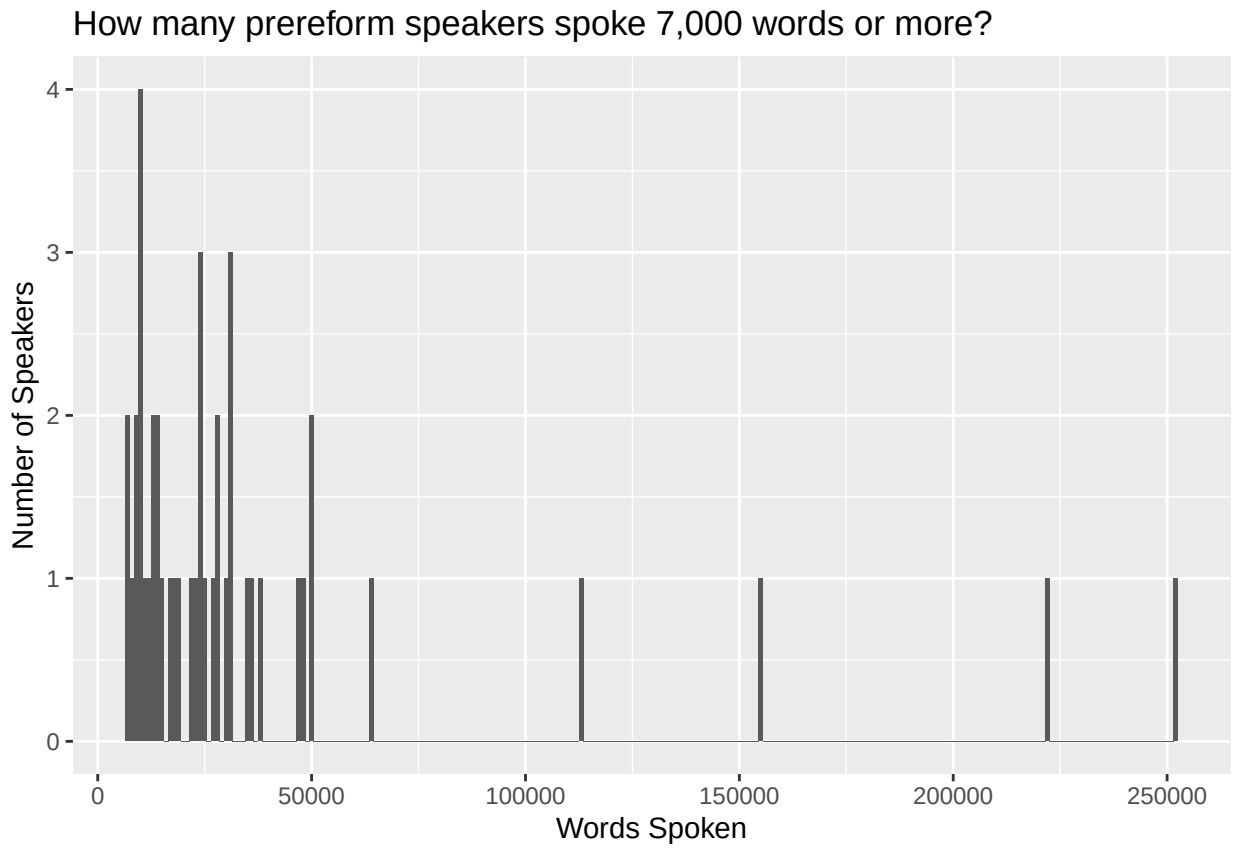
```r
# Visualize the distribution of high-volume speakers
# - geom_histogram() groups speakers into word-count bins
# - scale_x_continuous() formats the x-axis labels with commas
# - scale_y_continuous() formats the y-axis labels with commas
# - labs() adds the title and axis labels
ggplot(prereform_only_speakers_wordcount,
       aes(x = wordcount)) + # Plot word counts on the x-axis
  geom_histogram() + # Create a histogram
  scale_x_continuous(labels = comma) + # Format the x-axis labels
  scale_y_continuous(labels = comma) + # Format the y-axis labels
  labs(title = "Word Count Distribution: Prereform-Only Speakers", # Add a chart title
       x = "Word Count", # Label the x-axis
       y = "Number of Speakers") # Label the y-axis
```

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```




```r
# Visualize the distribution of speakers who spoke at least 7,000 words
# - geom_histogram() groups speakers into 1,000 word bins
```

```
# - labs() adds the title and axis labels
ggplot(prereform_only_speakers_wordcount,
       aes(x = wordcount)) + # Plot word counts on the x-axis
  geom_histogram(binwidth = 1000) + # Create 1,000 word bins
  labs(title = "How many prereform speakers spoke 7,000 words or more?", # Add a chart title
       x = "Words Spoken", # Label the x-axis
       y = "Number of Speakers") # Label the y-axis
```



Immediately after the reform of 1832, the shape of the histogram changes. The shape of the histogram is thicker in the middle. There are more speakers overall, of course.

Visualizations of this kind are useful for investigating the data – that is, getting answers to basic questions about how many people spoke and when. We can use them to inform further questions.

For example, we might be interested in whether the "frontbenchers" – the party leaders who spoke all the time – were any different prereform and postreform, or whether they look just like the aggregate story of all speakers we told below.

The histogram helps us to make informed decisions about where to put the cutoff for a "major speaker>' If the vast majority both before and after 1832 spoke less than 5000 words in the entire period (i.e., less than a ten-page paper), we might elect to look just at those few speakers who spoke over 10,000 words.

The code will also allow us to vary the cutoff number to see if doing so changes the outcome.

```
# Calculate the total words spoken by postreform-only speakers
# - filter() keeps only postreform only speakers
# - group_by() groups the data by speaker
# - summarize() adds the total words spoken by each speaker
postreform_only_speakers_wordcount <- all_speakers_annotated %>%
  filter(class == "postreform only") %>% # Keep postreform only speakers
  group_by(suggested_speaker) %>% # Group by speaker
  summarize(wordcount = sum(wordcount)) # Find the total words spoken
```

```
# Keep prereform only speakers who spoke at least 10,000 words
# - filter() keeps speakers who meet the word-count cutoff
major_prereform_speakers <- prereform_only_speakers_wordcount %>%
  filter(wordcount >= 10000) # Keep speakers who spoke at least 10,000 words

# Do the same for postreform only speakers who spoke at least 10,000 words
major_postreform_speakers <- postreform_only_speakers_wordcount %>%
  filter(wordcount >= 10000)

# Combine the major speakers from both periods
# - bind_rows() stacks the prereform and postreform speakers
major_speakers <- bind_rows(major_prereform_speakers,
                            major_postreform_speakers) # Combine the major speakers

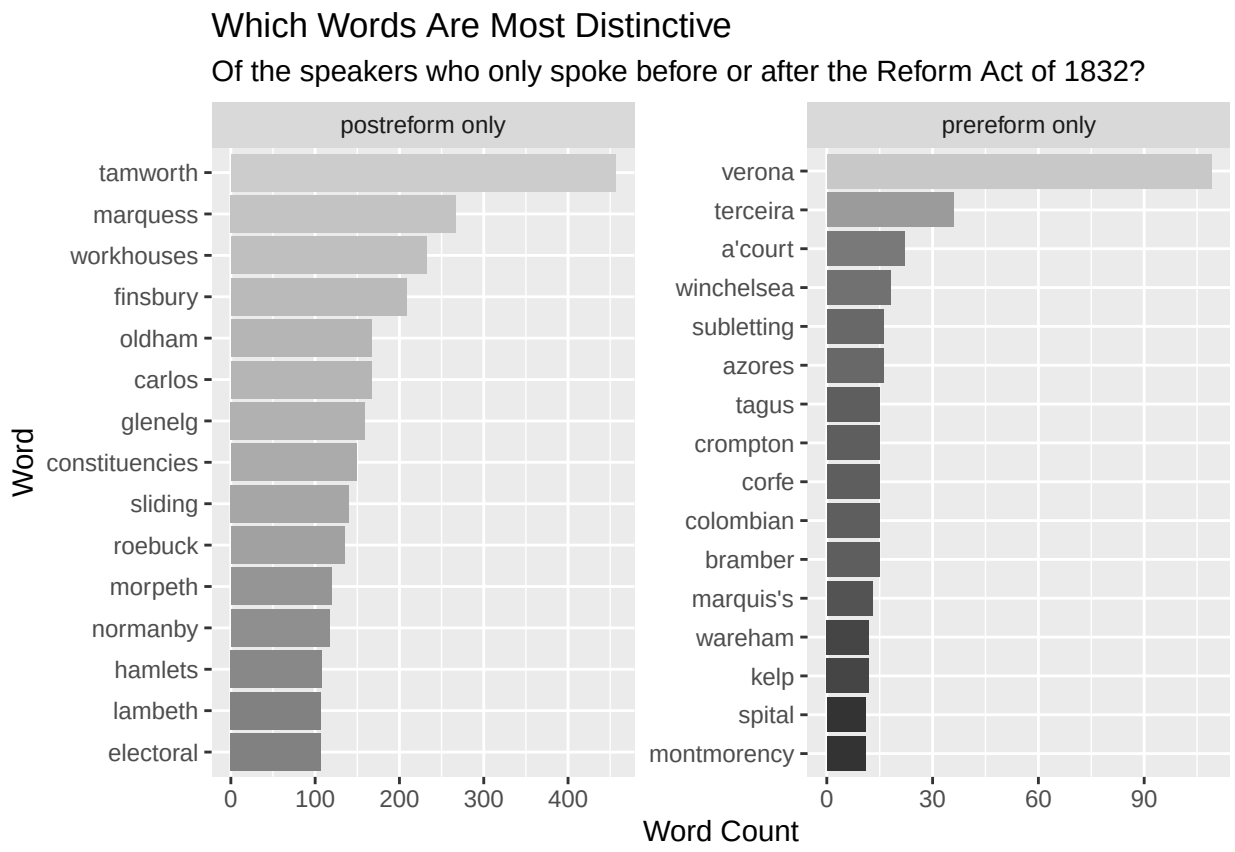
# Calculate tf-idf scores for words used by major speakers
# - filter() keeps only the major speakers
# - group_by() groups the words by speaker category
# - summarize() counts the words used by each category
# - bind_tf_idf() finds the tf-idf score for each word
top_words_w_speaker_classifications_tfidf <- words_w_speaker_classifications %>%
  filter(suggested_speaker %in% major_speakers$suggested_speaker) %>% # Keep only major speakers
  group_by(class, word) %>% # Group by speaker category and word
  summarize(words_per_period = n()) %>% # Count words by speaker category
  bind_tf_idf(word, class, words_per_period) # Find the tf-idf scores
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by class and word.
## i Output is grouped by class.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(class, word))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

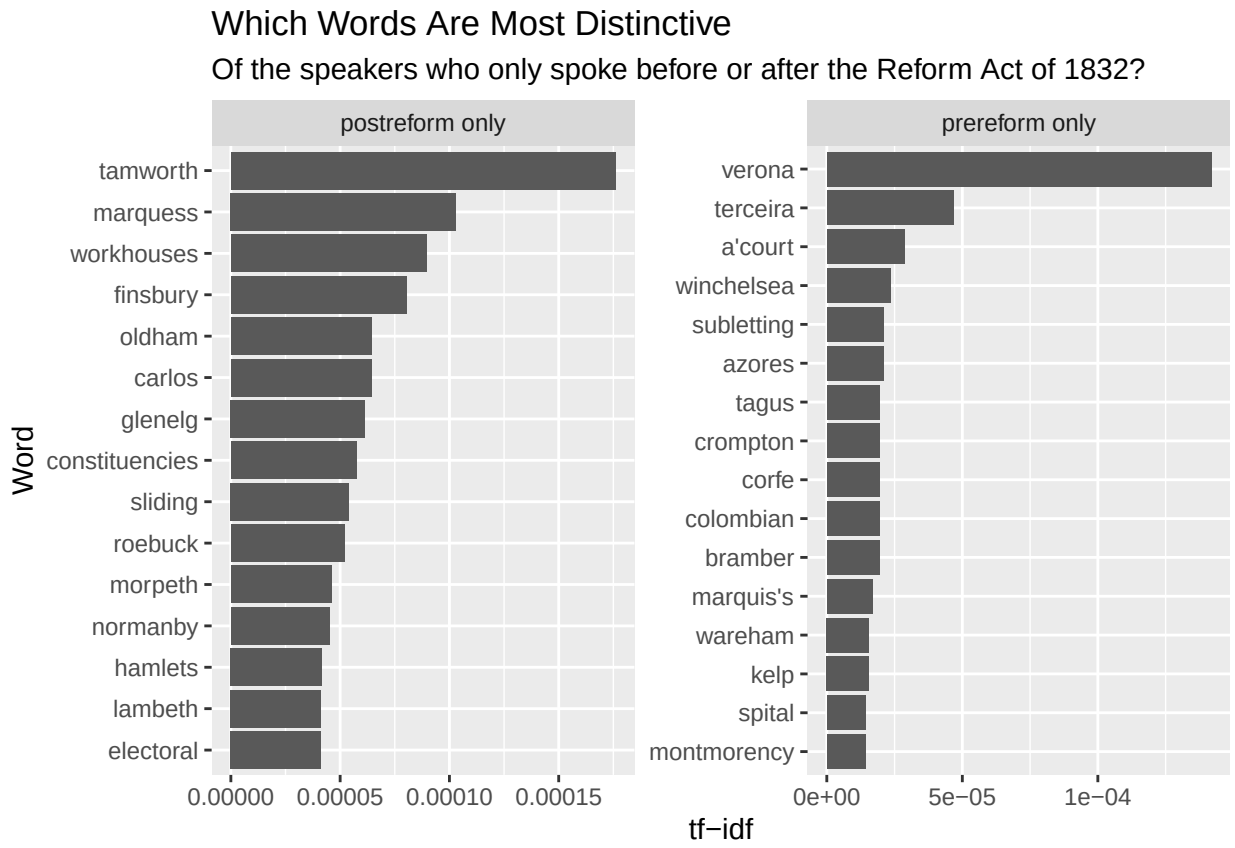
```
# Keep the most distinctive words for each speaker category
# - group_by() groups the words by speaker category
# - slice_max() keeps the 15 words with the highest tf-idf scores
# - ungroup() removes the grouping
```

```r
top_words_w_speaker_classifications_tfidf <- top_words_w_speaker_classifications_tfidf %>%
  group_by(class) %>% # Group by speaker category
  slice_max(order_by = tf_idf, n = 15) %>% # Keep the top 15 words
  ungroup() # Remove the grouping

# Visualize the most distinctive words used by major speakers
# - reorder_within() orders words by count separately within each speaker category
# - geom_col() creates a bar chart
# - facet_wrap() creates one panel for each speaker category
# - coord_flip() makes the word labels easier to read
# - scale_fill_grey() shades bars according to tf-idf score
# - scale_x_reordered() preserves word order within each panel
# - labs() adds the title, subtitle, and axis labels
ggplot(top_words_w_speaker_classifications_tfidf,
       aes(x = reorder_within(word, words_per_period, class), # Order words by count within each speake
           y = words_per_period, # Plot word counts on the y-axis
           fill = factor(tf_idf))) + # Shade bars by tf-idf score
  geom_col() + # Create a bar chart
  facet_wrap(~class, scales = "free") + # Create one panel per speaker category
  coord_flip() + # Flip the axes
  scale_fill_grey(guide = "none") + # Use grayscale shading and hide the legend
  scale_x_reordered() + # Preserve word order within each panel
  labs(title = "Which Words Are Most Distinctive", # Add a chart title
       subtitle = "Of the speakers who only spoke before or after the Reform Act of 1832?", # Add a sub
       x = "Word", # Label the x-axis
       y = "Word Count") # Label the y-axis
```




```
# Visualize the most distinctive words used by major speakers
# - reorder_within() orders words by tf-idf separately within each speaker category
# - geom_col() creates a bar chart
# - facet_wrap() creates one panel for each speaker category
# - coord_flip() makes the word labels easier to read
# - scale_x_reordered() preserves the word order within each panel
# - labs() adds the title, subtitle, and axis labels
ggplot(top_words_w_speaker_classifications_tfidf,
       aes(x = reorder_within(word, tf_idf, class), # Order words by tf-idf separately within each spea
           y = tf_idf)) + # Plot tf-idf scores on the y-axis
  geom_col() + # Create a bar chart
  facet_wrap(~class, scales = "free") + # Create one panel per speaker category
  coord_flip() + # Flip the axes
  scale_x_reordered() + # Preserve the word order within each panel
  labs(title = "Which Words Are Most Distinctive", # Add a chart title
       subtitle = "Of the speakers who only spoke before or after the Reform Act of 1832?", # Add a sub
       x = "Word", # Label the x-axis
       y = "tf-idf") # Label the y-axis
```



Some of the compelling words that elicit attention are geographical. How should we interpret the allusions to Verona, Australia, Persia, and Stockdale?

A final approach is to look at eaach time period on its own, freezing any other times. Such an approach allows us to ask, For their period, who is the most distinctive speaker?

```
# Calculate tf-idf scores for each speaker
# - filter() keeps only the major speakers
# - count() counts the words spoken by each speaker
# - bind_tf_idf() calculates the tf-idf score for each word
speaker_words_tfidf <- words_w_speaker_classifications %>%
  filter(suggested_speaker %in% major_speakers$suggested_speaker) %>% # Keep only major speakers
  count(class, suggested_speaker, word, name = "wordsperspeaker") %>% # Count words by speaker
  bind_tf_idf(word, suggested_speaker, wordsperspeaker) # Calculate the tf-idf scores

# Find the most distinctive speakers in each category
# - group_by() groups the data by speaker category and speaker
# - summarize() calculates each speaker's total tf-idf score and total words
# - slice_max() keeps the five speakers with the highest tf-idf scores
# - ungroup() removes the grouping
most_distinctive_speakers <- speaker_words_tfidf %>%
```

```
  group_by(class, suggested_speaker) %>% # Group by speaker category and speaker
  summarize(speaker_distinctiveness = sum(tf_idf), # Calculate each speaker's total tf-idf score
            total_words = sum(wordsperspeaker)) %>% # Calculate each speaker's total words
  group_by(class) %>% # Group by speaker category
  slice_max(speaker_distinctiveness, n = 5) %>% # Keep the five most distinctive speakers
  ungroup() # Remove the grouping

# Find the most distinctive words for each speaker
# - inner_join() keeps only the selected speakers
# - group_by() groups the words by speaker
# - slice_max() keeps the five words with the highest tf-idf scores
# - mutate() numbers the words within each speaker
# - ungroup() removes the grouping
most_distinctive_speakers_top_words <- speaker_words_tfidf %>%
  inner_join(most_distinctive_speakers, by = c("class", "suggested_speaker")) %>% # Keep the selected s
  group_by(class, suggested_speaker) %>% # Group by speaker category and speaker
  slice_max(tf_idf, n = 5) %>% # Keep the top five words
  mutate(word_number = row_number()) %>% # Number the words for each speaker
  ungroup() # Remove the grouping
```

```
# Display each speaker's five most distinctive words
# - select() keeps only the columns needed for the table
# - pivot_wider() places each distinctive word in a separate column
# - mutate() orders the speaker categories
# - arrange() sorts the rows by category and speaker
# - kable() formats the results as a table
most_distinctive_speakers_top_words %>%
  select(class, suggested_speaker, word_number, word) %>% # Keep the relevant columns
  pivot_wider(names_from = word_number, # Create one column for each word number
              values_from = word, # Fill the columns with words
              names_prefix = "Word ") %>% # Name the columns Word 1 through Word 5
  mutate(class = factor(class, levels = c("prereform only", "postreform only", "both periods"))) %>% #
  arrange(class, suggested_speaker) %>% # Sort by category and speaker
  kable(col.names = c("Period", "Speaker", "Word 1", "Word 2", "Word 3", "Word 4", "Word 5"), # Label t
        caption = "Most Distinctive Words by Speaker and Period. These are the speakers with more than
```

Table 6.3: Most Distinctive Words by Speaker and Period. These are the speakers with more than 7,000 words who had the most distinctive lexicons.

Period	Speaker	Word 1	Word 2	Word 3	Word 4	Word 5
prereform only	james_mackintosh_1886	spain	don	forgery	portugal	treaty
prereform only	joseph_yorke_1364	jackets	masts	bridge	seamen	waterloo
prereform only	michael_nolan_2187	removals	pauper's	occasional	parish	house's
prereform only	robert_wilson_2106	cuba	mexico	colombia	spain	mexican
prereform only	william_wilberforce_1181	slaves	slave	slavery	african	trinidad

Period	Speaker	Word 1	Word 2	Word 3	Word 4	Word 5
postreform only	henry_bruen_3303	carlow	woodcock	bruen	ejected	priest
postreform only	henry_rich_3512	executions	bode	assessors	affright	gallows
postreform only	james_buckingham_3161	impressment	seamen	india	buckingham	ships
postreform only	john_sinclair_5253	radicals	radical	jeffrey	sinclair	conservative
postreform only	william_ferrand_3687	league	truck	cloth	workpeople	stockport

The Marquis of Blandford was amongst the ultra-Tories who advocated the Reform Act as a way of containing Catholic power. The fact that he appears marked out for distinctiveness in the prereform era for his specific discussions of the intricacies of the vote ("burgess," "borough") as well as his engagement with radical advocates of reform ("tooke," an allusion to John Horne Tooke, the radical clergyman tried for treason in 1794) stands out.

We also see evidence of the passing of certain styles of engagement. After the reform act, the clergy who discussed church business ("diocess," "clergy," "diocesses," "communicant," "clerical") or divorce law ("divorce," "adultery," "marriage") like the Anglican Bishop of Limerick and Dr. Phillimore were heard from no more. Likewise, speakers oriented to the diplomacy on the continent of Europe ("a'court," "chateaubriand," "spain," "verona") like Mr. Macdonald disappeared, as did the voice of slaveholders like Mr. Marryat, a sloveholder who had opposed abolitionism.

The most distinctive stewards of the reformed parliament included representatives of the new industrial towns ("cloth," "mill," "stockport," "corn," "wool," "export") like Mr. Ferrand and Mr. Cayley. Similar were Mr. Buckingham, who criticized the East India Company, advocated for the freedom of the press and the abolition of slavery, and urged the end of flogging for sailors; we learn from the measure of distinctiveness that the cause of "impressment" and issues in "india" distinguished him from his peers more than his advocacy of the press or abolition. We also see a portrait of the new face of conservatism post-1832, embodied by Mr. James Emerson Tennent, who attempted to defend the textile industry in Ireland through the advocacy of copyright laws for designs for printed fabric, and Sir George Sinclair – who was ahead of his time in using the language of "conservative" rather than "Tory."

The portrait of change we have here is exceptional rather than the rule. We're looking at the voices who stood out from the pack before and after reform – at the boundaries for discourse made available by a new form of democracy – rather than the causes endorsed by the party leaders. This is a different picture of reform than that available to us in traditional histories like that of Dame Antonia Frasier, who reads the diaries of court commentators and society hostesses for commentary on a string of prime ministers. To draw a contemporary metaphor, our method allows us to read for the exceptional – for the Jeremy Corbyn and Nigel Farage or Ted Cruz and Alexandra Ocasio-Cortez as indexes of the political variations of the nineteenth century, rather than for history as told through the memoirs of the Boris Johnsons, Theresa Mays, Donald Trumps and Joe Bidens. It is not necessarily a portrait of power – of what was executed. But as a portrait of belief, ideas, and enthusiasms, it is necessarily more complete.

3 Exercises

1) This chapter builds on previous chapters, expanding what can be demonstrated with the tools provided so far. A great deal of space has been spent on creating elegant and informative visualizations, for example, timelines labeled with names that accurately reflect how an individual was described in the text, even while metadata behind the scenes ensures that we count one person per one dot. Why is it important to reference the names found in the original text? Why is it important to use metadata to ensure that one person gets one dot?

2) In this chapter we annotate our data. Annotation is important because it allows us to add meaning to otherwise difficult to interpret visualizations. How might the way we choose to annotate our data be a scholarly choice in its own right? In what ways can annotation shape the kinds of historical questions we are able to ask or how we can interpret the past?

3) Why is performing observational validation an important step when processing and analyzing data? What kinds of problems can it help researchers detect? How frequently should researchers return to the data to validate their results as they move through different stages of analysis? When examining the data during validation, what kinds of patterns, anomalies, or errors should they be looking for?

4) The approach profiled here generated many analyses filled with obscure names, in part because we have used the stricter tf-idf rather than the more inclusive log likelihood measure. For the analyses of prereform and post reform speakers and their words, change the code so that the analysis uses log likelihood. Are the results any easier to interpret?

5) This chapter leverages tf-idf to ask, in different ways, what is special about the language before and after the Reform Act? In your own words, explain what was done with the data in each of these exercises:

a) identifying how many words were spoken prereform and postreform
b) comparing speakers from prereform, postreform, and both periods
c) distinctiveness before and after the Reform act of 1832

5) In Table 6.1, the top ten most frequently used words in Parliament before and after the Reform Act are quite similar. To make the analysis more meaningful, we used tf-idf to identify words that are more distinctive to each period. What are some other metrics or approaches an analyst could use at this stage to highlight meaningful differences in language between the two periods?

6) Tf-idf can draw attention to infrequent words, including names that appear less commonly in the corpus, such as Quamina Gladstone. Why is it valuable for researchers to notice these uncommon names or references? How might identifying such figures lead historians to new questions, overlooked people or events, or important historical connections?

7) As mentioned in the chapter, historians often argue that after the Reform Act of 1832 Parliament began focusing more on issues affecting the growing middle class and urban population. Has text mining tested, validated, or complicate this interpretation?

8) The approach profiled here generated many analyses filled with obscure names, in part because we have used the stricter tf-idf rather than the more inclusive log likelihood measure. For the analyses of prereform and post reform speakers and their words, change the code so that the analysis uses log likelihood. Are the results any easier to interpret?